

Structures de données et algorithmes

Irène Charon, Olivier Hudry

Septembre 2007

Département informatique et réseaux
École nationale supérieure des télécommunications

Table des matières

1. Notions d'algorithme et de complexité	1
1.1. Algorithme	1
1.2. Complexité	3
2. Premières structures de données	5
2.1. Introduction	5
2.2. Structures linéaires	6
2.2.1 Listes	6
2.2.2 Piles	8
2.2.3 Files	10
2.3. Arbres	12
2.3.1 Arbres généraux	12
2.3.2 Parcours d'un arbre général	14
2.3.3 Arbres binaires	15
3. Recherche et tri	20
3.1. Introduction	20
3.2. Recherche dichotomique dans un tableau trié	20
3.3. Complexité d'un algorithme de tri	21
3.4. Tri sélection	23
3.5. Tri insertion	24
3.6. Tri rapide	25
3.7. Arbre binaire de recherche	28
3.8. Structure de tas et tri tas	30
3.8.1 Définition	30
3.8.2 Intérêt de la structure d'arbre parfait	31
3.8.3 Exemple d'insertion d'un nouvel élément dans un tas	32
3.8.4 Pseudo-code de l'insertion d'une donnée dans un tas	33
3.8.5 Complexité d'une insertion dans un tas	34
3.8.6 Principe du tri tas	34
3.8.7 Exemple de suppression de la racine	34
3.8.8 Pseudo-code de la descente d'une donnée	35

3.8.9 Complexité de la descente de la donnée de la racine.....	36
3.8.10 Pseudo-code du tri tas	36
3.8.11 Complexité du tri tas	37
3.8.12 Exercice.....	37
4. Le hachage.....	38
4.1. Principe général	38
4.2. Exemple pour illustrer le risque de collisions	38
4.3. Exemples de fonctions de hachage	39
4.4. Le hachage linéaire	40
4.5. Le hachage avec chaînage interne	41
4.6. Le hachage avec chaînage externe.....	42
4.7. Nombre de comparaisons.....	43
5. L'algorithme de Huffman	45
5.1. Présentation du problème	45
5.2. Quelques remarques préliminaires	46
5.3. Description de l'algorithme de Huffman	49
5.4. Exemple d'application	50
Index	53
Bibliographie	54

1. Notions d'algorithme et de complexité

1.1. Algorithme

Un *algorithme* est une suite finie d'instructions non ambiguës pouvant être exécutées de façon automatique. Un algorithme prend en entrée des données et produit un résultat.

Nous verrons de nombreux exemples d'algorithmes. On peut dire qu'une recette de cuisine est un algorithme, l'entrée étant les ingrédients et la sortie le plat cuisiné. Nous décrirons aussi, par exemple, des algorithmes pour trier des éléments d'un ensemble totalement ordonné.

On peut distinguer trois façons de décrire un algorithme.

- Une première méthode consiste à en donner le principe.

Exemple : pour détecter si un certain élément figure dans une liste non triée, le principe d'un algorithme de recherche séquentielle consiste à : comparer successivement tous les éléments de la liste avec l'élément recherché jusqu'à rencontrer l'élément recherché ou bien atteindre la fin de la liste.

- Une deuxième méthode consiste à utiliser du pseudo-code ; il s'agit d'une façon d'exprimer l'algorithme en précisant la façon de ranger les données, les variables, les initialisations et en séparant les différentes instructions ; on utilise un certain formalisme pour exprimer les affectations, les instructions conditionnelles, les boucles, etc. Il y a différents formalismes possibles mais, après en avoir choisi un, il est très préférable de s'y tenir.

Exemple : le même algorithme que dans l'exemple précédent peut être décrit comme ci-dessous.

Recherche séquentielle dans un tableau

Il y a n données.

Les données se trouvent dans un tableau T à partir de l'indice 1.

La donnée à rechercher est notée *clé*.

On utilise une variable booléenne notée *trouvé* et un entier i .

- $i \leftarrow 1$;
- $\text{trouvé} \leftarrow \text{faux}$;
- tant que ((non *trouvé*) et ($i \leq n$))
début boucle tant que
 - si ($T[i] = \text{clé}$), alors $\text{trouvé} \leftarrow \text{vrai}$;
 - sinon $i \leftarrow i + 1$;
 fin boucle tant que
- renvoyer *trouvé*.

Entre autres choix, le formalisme ci-dessus utilise le symbole « = » pour le test d'égalité et le symbole « $\leftarrow$ » pour l'affectation ; cette dernière signifie que la valeur du membre de droite est transmise à la variable du membre de gauche.

On peut éventuellement supprimer les indications telles que « début boucle tant que » et « fin boucle tant que » en s'appuyant sur l'indentation, à condition que celle-ci soit suffisamment claire. C'est ce que nous ferons par la suite.

- Une troisième méthode consiste à traduire l'algorithme dans un langage de programmation.

Les trois méthodes décrites ci-dessus sont de plus en plus précises, de moins en moins ambiguës. Malheureusement, elles sont aussi de moins en moins faciles à lire et à comprendre. Si on veut programmer un algorithme, il sera généralement pertinent d'enchaîner les trois méthodes : d'abord en donner le principe, puis écrire le pseudo-code, avant de passer à la programmation.

Remarquons que, lorsqu'on propose un algorithme, il est généralement nécessaire de le prouver, c'est-à-dire de prouver qu'il fait bien, et dans tous les cas, ce qu'on attend de lui. Il existe des techniques pour effectuer des preuves d'algorithme, techniques que nous ne développerons pas dans ce polycopié.

Implémenter un algorithme signifie le traduire dans un langage de programmation pour pouvoir l'exécuter à l'aide d'un ordinateur. Pour effectuer l'implémentation, il faut choisir la façon de représenter les données, c'est-à-dire choisir la structure des données ; nous développerons différentes possibilités de structures de données dans les chapitres suivants. Pour résoudre un problème, il y a souvent de nombreux algorithmes possibles et, pour chaque algorithme, plusieurs choix de structures de données. Un algorithme assorti d'une structure des données pour son implémentation peut avoir plusieurs caractéristiques :

- *Sa simplicité* : un algorithme simple est facile à comprendre, à implémenter et généralement à prouver.
- *La qualité du résultat obtenu* : par exemple, une recette pour faire une mousse au chocolat peut donner un résultat plus ou moins agréable à déguster.
- *Le temps pris pour effectuer l'algorithme* : là encore, cette caractéristique dépend à la fois de l'algorithme et des structures de données retenues. Il n'est pas matériellement possible de résoudre un problème à l'aide d'un algorithme

nécessitant quelques milliards d'années pour s'exécuter. On peut aussi préférer un algorithme qui s'exécutera en une seconde plutôt qu'en une heure. De plus, la ressource temps d'un ordinateur est souvent précieuse et on cherche alors à l'économiser. On verra que, d'un algorithme à l'autre, et d'une structure de données à l'autre, les temps d'exécution peuvent être très différents. Cette caractéristique de temps d'exécution s'exprime avec ce qu'on appelle la *complexité* (en temps) de l'algorithme.

- *La place prise en mémoire* (complexité en place) : cette place va dépendre à la fois de l'algorithme et des structures de données retenues ; il ne faut pas dépasser les capacités de la mémoire de la machine sur laquelle l'algorithme s'exécutera et, si la place prise en mémoire est importante, on peut être conduit à limiter la taille des problèmes considérés.

Dans la suite, quand nous parlerons de complexité, il s'agira toujours de complexité en temps.

Les relations entre algorithmes et structures de données sont variables ; la plupart des algorithmes, dès la formulation de leur principe, s'appuient sur une certaine structure de données. Parmi les exemples que nous verrons plus tard, le tri rapide s'appuie sur une structure linéaire des données, le tri par arbre binaire de recherche sur une structure d'arbre binaire de recherche, le tri tas sur une structure de tas, les algorithmes de graphes sur une structure selon des graphes... Il ne reste souvent qu'à préciser la structure de données pour passer de l'algorithme à son implémentation.

Il arrivera fréquemment que, entre deux structures de données, l'une soit meilleure en ce qui concerne la place prise en mémoire alors que l'autre est meilleure en ce qui concerne la complexité.

1.2. Complexité

Étant donné un algorithme, nous appelons *opérations élémentaires* :

- un accès en mémoire pour lire ou écrire la valeur d'une variable ou d'une case d'un tableau ;
- une opération arithmétique entre entiers ou entre réels : addition, soustraction, multiplication, division, calcul du reste dans une division entière ;
- une comparaison entre deux entiers ou deux réels.

Exemple : si on considère l'instruction $c \leftarrow a + b$, on peut compter quatre opérations élémentaires :

- l'accès en mémoire pour lire la valeur de a ,
- l'accès en mémoire pour lire la valeur de b ,
- l'addition de a et b ,
- l'accès en mémoire pour écrire la nouvelle valeur de c .

Pour un problème donné, on appelle *instance* tout jeu de données de ce problème. Par exemple, pour un problème de tri, on obtient une instance en spécifiant la valeur numérique des nombres à trier.

Soient f et g deux fonctions positives d'une même variable entière n (resp. de deux mêmes variables entières n et m , ou plus). La fonction f est dite avoir un ordre de grandeur au plus égal à celui de la fonction g s'il existe un entier strictement positif k et un entier N (resp. deux entiers N et M) tels que, pour tout $n \geq N$ (resp. $n \geq N$ et $m \geq M$), on ait $f(n) \leq k g(n)$ (resp. $f(n, m) \leq k g(n, m)$) ; on écrira $f = O(g)$ (notation de Landau). Les deux fonctions sont dites avoir même ordre de grandeur si l'ordre de grandeur de l'une est au moins égal à l'ordre de grandeur de l'autre et réciproquement ; on pourra écrire : $f = \Theta(g)$. Par exemple, les fonctions $f(n) = 3n^2 - 5n + 4$ et $g(n) = n^2$ ont même ordre de grandeur. On dira aussi que g est un ordre de grandeur de f .

On considère un algorithme $\mathcal{A}$. On appelle *complexité de $\mathcal{A}$* tout ordre de grandeur du nombre d'opérations élémentaires effectuées pendant le déroulement de l'algorithme. On exprime ce nombre d'opérations en fonction de paramètres associés aux instances à traiter ; on pourra par exemple exprimer la complexité d'un tri en fonction du nombre de données à trier. Néanmoins, il se peut qu'avec deux jeux de données différents correspondant aux mêmes paramètres, la complexité ne soit pas la même. Par exemple, un algorithme de tri pourra être plus rapide s'il s'agit de trier des données déjà triées que s'il s'agit de données très désordonnées. On peut alors s'intéresser à :

- la complexité *dans le pire des cas* : les paramètres avec lesquels on exprime la complexité étant fixés, on considère le plus grand nombre d'opérations élémentaires effectuées sur l'ensemble des instances correspondant à ces paramètres ; on cherche ainsi un majorant de la complexité qui puisse être atteint dans certains cas ; c'est ce qu'on fait le plus généralement ;
- la complexité *dans le meilleur des cas* : les paramètres avec lesquels on exprime la complexité étant fixés, on considère le plus petit nombre d'opérations élémentaires effectuées sur l'ensemble des instances correspondant à ces paramètres ; cette complexité peut venir compléter la précédente mais ne sera jamais suffisante pour l'utilisateur ;
- la complexité *en moyenne* : les paramètres avec lesquels on exprime la complexité étant fixés, il faut alors faire la moyenne des nombres d'opérations élémentaires effectuées, moyenne portant sur tous les jeux de données correspondant à ces paramètres. Ce calcul est généralement difficile et souvent même délicat à formuler car il faut connaître la probabilité de chacun des jeux de données pour effectuer un calcul pertinent de cette moyenne.

Dans certains cas, on peut être amené à ne calculer qu'un majorant (qu'on essaie de rendre le plus petit possible) de la complexité d'un algorithme.

Prenons le cas de la recherche d'un élément dans une liste non triée de n éléments avec l'algorithme développé dans la section 1.1. Le pire des cas est celui où la donnée ne figure pas, où la complexité est de l'ordre de n . Le meilleur des cas est celui où l'élément recherché est le premier de la liste, cas où la complexité est de l'ordre de 1. Si maintenant on veut faire une moyenne sur tous les jeux de données à n éléments et tout élément recherché, on peut supposer que, dans le cas où l'élément recherché figure dans la liste, toutes les places sont équiprobables ; mais quelle est la probabilité que l'élément recherché figure ? On peut la poser comme étant égal à 0,5, mais c'est arbitraire. Néanmoins, pour l'exemple, très simple, quelle que soit la façon de calculer la moyenne, on obtient un ordre de n ; on dira que cet algorithme est *linéaire* en moyenne.

2. Premières structures de données

2.1. Introduction

Les *structures de données* spécifient la façon de représenter les données du problème considéré ; cela est nécessaire en particulier pour un traitement par un algorithme à l'aide d'un ordinateur.

Deux préoccupations principales interviendront dans le choix d'une telle structure : la place qu'elle consomme dans la mémoire de l'ordinateur et la facilité qu'elle offre quand on cherche à accéder à une certaine donnée. Les structures de données sont définies indépendamment du langage de programmation qui interviendra dans l'écriture finale du programme qui les manipulera ; on supposera néanmoins que ce langage offre les outils nécessaires (par exemple les pointeurs) pour définir et manipuler ces structures de données. La plupart des langages de programmation permettent d'attribuer et d'utiliser la mémoire disponible de différentes façons :

- sous forme de *variables* isolées les unes des autres dans la mémoire de la machine ; elles peuvent être d'un type prédéfini par le langage (le type entier, le type réel, etc.) ou d'un type défini par l'utilisateur à partir des types prédéfinis précédents, par exemple des types conçus pour décrire des variables comprenant plusieurs données hétérogènes, type nommé *structure* dans le langage C ou *enregistrement* dans d'autres langages ; nous appellerons *structure* une variable appartenant à un type comportant plusieurs composantes (hétérogènes ou non) et nous appellerons *champs* les composantes d'une structure ;
- sous forme de *tableaux*, c'est-à-dire d'une suite de variables de même type associées à des « cases » consécutives dans la mémoire ; cette « consécutivité » permet à l'ordinateur de savoir où sont rangés les éléments du tableau, ce qui permet d'avoir accès directement à une variable du tableau à partir d'un indice donné.
- à l'aide de *pointeurs* ou *adresses* ou *références*, indiquant la localisation dans la mémoire de l'objet auquel on s'intéresse.

Dans certains cas (variables isolées, tableaux statiques), la place en mémoire est attribuée par le compilateur et ne peut donc pas être modifiée au cours du programme. Dans d'autres cas (tableaux dynamiques, listes chaînées), l'attribution de la mémoire nécessaire est effectuée pendant le déroulement du programme et peut donc varier pendant celui-ci.

Les structures de données classiques appartiennent le plus souvent aux familles suivantes :

- les *structures linéaires* : il s'agit essentiellement des structures représentables par des

listes linéaires, c'est-à-dire des tableaux ou des listes chaînées unidimensionnelles ; on y trouve en particulier les *pires*, pour lesquelles les données sont ajoutées ou supprimées à partir d'une même extrémité, et les *files*, pour lesquelles les données sont ajoutées à une extrémité tandis qu'elles sont supprimées à l'autre ;

- les *structures arborescentes*, avec la sous-famille importante des *arbres binaires* ;
- les *structures relationnelles* : celles-ci prennent en compte des relations existant ou n'existant pas entre les entités qu'elles décrivent ; les relations binaires sont représentables sous forme de *graphes* (orientés ou non).

2.2. Structures linéaires

Les structures linéaires tirent leur nom du fait que les données y sont organisées sous forme d'une liste dans laquelle elles sont mises les unes derrière les autres. On peut représenter une telle liste à l'aide d'un tableau unidimensionnel ou sous la forme d'une liste chaînée. Selon la nature des opérations autorisées, on obtient différents types de listes, en particulier les piles et les files.

2.2.1 Listes

Une liste est une suite ordonnée d'éléments d'un type donné ; une liste peut contenir zéro, un ou plusieurs éléments.

Exemples :

- (3 , 7, 2, 8, 10, 4) est une liste d'entiers ;
- (lundi, mardi, mercredi, jeudi, vendredi, samedi, dimanche) est la liste des jours de la semaine.

La *longueur* d'une liste est son nombre d'éléments. Une liste *vide* est une liste de longueur nulle. La *tête* de la liste est son premier élément. La définition de la *queue* d'une liste n'est pas universelle ; certains la définissent comme étant le dernier élément de la liste et d'autres comme la liste obtenue en enlevant la tête.

On s'intéresse souvent à des listes d'éléments appartenant à un ensemble sur lequel existe une relation d'ordre total (entiers, chaînes de caractères, objets identifiés par un entier...). C'est le cas quand on cherche à trier les éléments de la liste, c'est-à-dire à permuter les éléments de la liste pour qu'ils deviennent placés par ordre croissant (ou décroissant).

Opérations sur les listes

Les opérations habituelles sur les listes sont les suivantes.

- *Insertion* d'un élément au début, à la fin, ou entre deux éléments de la liste. En particulier, on pourra chercher à insérer un élément dans une liste triée à une bonne place, c'est-à-dire en conservant le fait que la liste est triée.
- *Suppression* d'un élément quelconque.

- *Recherche* : cette opération peut renvoyer *vrai* ou *faux* selon qu'un élément donné figure ou non dans la liste ; elle peut aussi renvoyer l'adresse de l'élément recherché sous forme selon les cas d'un indice d'un tableau ou d'un pointeur.
- *Concaténation* de deux listes $L1$ et $L2$ contenant des éléments d'un même type : cette opération donne la liste obtenue en mettant d'abord les éléments de $L1$ puis ceux de $L2$.

On pourrait ajouter l'opération de *tri*, mais nous consacrerons un chapitre entier à ce sujet, que nous laissons pour l'instant de côté.

Implémentation d'une liste par un tableau

On peut représenter une liste par un tableau. L'indice du début de la liste sera alors fixé (en général, ce sera l'indice 0 ou l'indice 1). Il faudra connaître et actualiser le cas échéant le nombre n d'éléments de la liste pour savoir d'où à où les éléments se trouvent dans le tableau.

Si on veut insérer un nouvel élément dans la liste :

- s'il s'agit d'insérer un élément à sa place dans une liste triée, il faudra décaler tous les éléments suivants d'une case, auquel cas la complexité dans le pire des cas est de l'ordre de la longueur de la liste ;
- si l'ordre des éléments n'importe pas, on insérera en général l'élément nouveau à la fin du tableau, la complexité étant alors de l'ordre de 1.

Si on veut supprimer un élément de la liste :

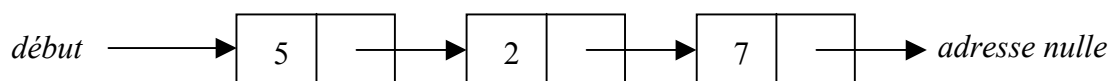
- s'il s'agit de supprimer un élément d'une liste triée, il faudra décaler tous les éléments suivants, d'où une complexité au pire de l'ordre de la longueur de la liste ;
- si l'ordre des éléments n'importe pas, le plus rapide sera de mettre à la place de l'élément à supprimer le dernier élément de la liste.

Si maintenant on veut concaténer deux listes $L1$ et $L2$, soit on recopie $L1$ puis $L2$ dans un nouveau tableau, soit on recopie les éléments de $L2$ à la suite de ceux de $L1$ dans le tableau qui contenait $L1$; dans les deux cas, il faut évidemment que les tableaux soient suffisamment grands. La complexité est soit de l'ordre du nombre total d'éléments de $L1$ et $L2$, soit de l'ordre du nombre d'éléments de $L2$.

Un inconvénient d'un tableau, par rapport à la liste chaînée que nous développons ci-dessous, est qu'il faut prendre garde à ne pas déborder du tableau. Quand le tableau est plein, il faut soit refuser les insertions, soit prendre un tableau plus grand et y recopier la liste.

Implémentation d'une liste par une liste chaînée

Une liste chaînée est constituée de « maillons » ; les maillons peuvent contenir des données et l'adresse d'un autre maillon. Les données contenues par les maillons sont du même type d'un maillon à l'autre. Une liste chaînée composée des entiers 5, 2 et 7 pourra être représentée comme ci-dessous :



où *début* est l'adresse d'un maillon.

Le maillon qui se trouve à l'adresse *début* contient l'élément qui constitue la tête de la liste.

Pour accéder à l'ensemble des éléments de la liste, on part de l'adresse *début*. On peut avec cette adresse accéder au premier maillon. On connaît ainsi le premier élément de la liste et on lit l'adresse contenue dans ce premier maillon, qui est celle du deuxième maillon ; on peut alors accéder au deuxième maillon pour connaître le deuxième élément de la liste et ainsi de suite. Il est nécessaire qu'une adresse spéciale soit mise dans le dernier maillon de la liste pour qu'on puisse savoir qu'il s'agit du dernier maillon. Cette adresse spéciale est prévue par les langages de programmation qui permettent de manipuler les listes chaînées ; elle s'appelle en général *nil*, ou *NULL*, ou *null* ; c'est toujours en quelque sorte l'adresse qui vaut zéro, notée *adresse nulle* sur notre schéma.

À condition de connaître certaines adresses appropriées, on constate que l'insertion et la suppression d'un élément, ou bien la concaténation de deux listes se fait avec une complexité de l'ordre de 1. En revanche, nous verrons dans le chapitre suivant que la recherche d'un élément dans une liste triée de n éléments peut se faire avec une complexité de l'ordre de $\ln(n)$ lorsqu'on utilise un tableau, alors que la complexité est toujours dans le pire des cas et en moyenne de l'ordre de n lorsque la liste est codée avec une liste chaînée.

La gestion informatique d'une liste chaînée se fait généralement en employant une allocation dynamique de mémoire chaque fois qu'un nouveau maillon est nécessaire ; les adresses sont alors des adresses dans la mémoire principale de l'ordinateur. Néanmoins, on peut gérer une structure de liste chaînée avec un tableau de maillons, les adresses de maillons devenant alors des indices du tableau ; avec cette dernière façon de faire, on retrouve l'inconvénient de la limitation du nombre de données au nombre de cases du tableau de maillons. Dans la suite, nous supposerons implicitement qu'on utilise une allocation dynamique de mémoire pour chaque nouveau maillon.

Dans le pseudo-code plus bas, on notera *rien* l'adresse nulle ; si on considère un maillon d'adresse p , on notera (comme en langage Java) $p.donnée$ la partie du maillon qui contient la donnée et $p.suivant$ la partie du maillon qui contient l'adresse du maillon suivant (en langage C, on note cela $p \rightarrow donnée$ et $p \rightarrow suivant$).

2.2.2 Piles

Une *pile* (*stack* en anglais) est une liste dans laquelle l'insertion ou la suppression d'un élément s'effectue toujours à partir de la même extrémité de la liste, extrémité appelée le *début* de la pile. L'action consistant à ajouter un nouvel élément au début de la pile s'appelle *empiler* ; celle consistant à retirer l'élément situé au début de la pile s'appelle *dépiler*. Une pile permet de modéliser un système régi par la discipline « dernier arrivé - premier sorti » ; on dit souvent qu'il s'agit d'un traitement LIFO (*last in, first out*). La *hauteur* de la pile est son nombre d'éléments.

Les fonctions souvent définies dans l'implémentation d'une pile sont (les fonctions 1, 3 et 4 font partie de toute implémentation d'une pile) :

1. répondre à la question : la pile est-elle vide ?
2. éventuellement, selon l'implémentation, répondre à la question : la pile est-elle pleine ?
3. empiler un élément, en traitant éventuellement le cas où la pile serait pleine

(fonction souvent appelée *push* en anglais),

4. dépiler un élément, en traitant le cas où la pile serait vide (fonction souvent appelée *pop* en anglais),
5. éventuellement, donner la valeur de l'élément qui se trouve au début de la pile, sans dépiler celui-ci,
6. éventuellement, vider la pile.

Implémentation d'une pile par un tableau

Si la pile est représentée par un tableau T , on doit gérer une variable pour la hauteur de la pile, variable que nous notons *hauteur*.

Supposons que les éléments de la pile soient stockés à partir de l'indice 0 (resp. 1) du tableau ; l'empilement d'un nouvel élément x consiste alors à introduire x dans la case $T(\text{hauteur})$ (resp. $T(\text{hauteur} + 1)$) et à remplacer *hauteur* par *hauteur* + 1 ; le dépilement consiste au contraire à supprimer l'élément $T(\text{hauteur} - 1)$ (resp. $T(\text{hauteur})$) de T puis à remplacer *hauteur* par *hauteur* - 1.

La pile est vide si la variable *hauteur* vaut zéro.

La pile est pleine si la hauteur est égale à la dimension du tableau.

Pseudo-code

On suppose qu'on utilise un tableau à N cases indicées de 1 à N .

On utilise une variable *hauteur*. La pile est vide si et seulement si *hauteur* vaut 0, la pile est pleine si et seulement si la *hauteur* vaut N . La variable *hauteur* est initialisée à 0.

Pour la fonction *empiler*, on choisit ci-dessous de ne rien faire si on appelle la fonction *empiler* alors que la pile est pleine. Dans un code plus élaboré, ce cas devrait être traité.

empiler(clé)

Si *hauteur* < N , alors

- *hauteur* $\leftarrow$ *hauteur* + 1;
- $T[\text{hauteur}] \leftarrow \text{clé}$;

Pour la fonction *dépiler*, on choisit ci-dessous de ne rien faire si on appelle la fonction *dépiler* alors que la pile est vide. Dans un code plus élaboré, ce cas devrait être traité.

dépiler

Si *hauteur* > 0, alors

- *clé* $\leftarrow T[\text{hauteur}]$;
- *hauteur* $\leftarrow$ *hauteur* - 1;
- renvoyer *clé*;

Implémentation d'une pile par une liste chaînée

Si la pile est représentée par une liste chaînée à laquelle on accède par un pointeur *début*, on ajoute un nouvel élément *clé* de la manière suivante. On crée un nouveau maillon *M* destiné à contenir *clé* et on y insère *clé* dans le champ de *M* prévue à cet effet. Dans le champ de *M* destiné à contenir l'adresse du maillon suivant de la liste après insertion de *clé*, on met la valeur du pointeur *début*. Enfin, pour accéder à la nouvelle liste à l'aide du pointeur *début*, on attribue à *début* l'adresse de *M*. De la même façon, pour dépiler un élément, on extrait l'élément contenu dans le premier maillon de *L* (celui auquel on accède grâce à *début*) puis on décale *début* en le faisant pointer sur le maillon qui suit le premier maillon ; on peut alors en général libérer l'espace en mémoire alloué au maillon contenant *clé*.

La pile est vide si l'adresse *début* est égale à l'adresse nulle (voir plus haut).

La pile n'est jamais pleine (il faut néanmoins ne pas dépasser la capacité en mémoire de la machine sur laquelle on travaille, ce qui peut se produire en particulier si on empile et qu'on dépile fréquemment et qu'on oublie de désallouer l'espace en mémoire quand on dépile un élément).

Pseudo-code

On utilise une variable *début*. La pile est vide si et seulement si *début* vaut *rien*. La variable *début* est initialisée à *rien*. Une fonction nommée *nouvelle_cellule* permet de créer une cellule et retourne l'adresse de celle-ci.

empiler(*clé*)

- $p \leftarrow \text{nouvelle_cellule};$
- $p.\text{donnée} \leftarrow \text{clé};$
- $p.\text{suivant} \leftarrow \text{début};$
- $\text{début} \leftarrow p;$

Pour la fonction *dépiler*, on choisit ci-dessous de ne rien faire si on appelle la fonction *dépiler* alors que la pile est vide. Dans un code plus élaboré, ce cas devrait être traité.

dépiler

Si *début* $\neq$ *rien*, alors

- $\text{clé} \leftarrow \text{début.donnée};$
- $\text{début} \leftarrow \text{début.suivant};$
- renvoyer *clé*;

Dans cette dernière fonction, il peut convenir, selon le langage de programmation, de libérer l'espace-mémoire qui était alloué à la cellule contenant la donnée qui a été dépilée.

2.2.3 Files

Une *file* est une liste dans laquelle toutes les insertions de nouveaux éléments s'effectuent d'un même côté de la liste appelé *fin* et toutes les suppressions d'éléments s'effectuent toujours à partir de l'autre extrémité, appelée *début*. L'action consistant à ajouter un nouvel élément à la fin de la file s'appelle *enfiler* ; celle consistant à retirer l'élément situé au début

de la file s'appelle *défiler*. Une file permet de modéliser un système régi par la discipline « premier arrivé - premier sorti » ; on dit souvent qu'il s'agit d'un traitement FIFO (*first in, first out*).

Implémentation d'une file par un tableau

Si la file est représentée par un tableau T , on doit gérer deux indices *début* et *fin* indiquant les cases de T entre lesquelles se trouvent toutes les données ; la donnée devant sortir en premier figure dans la case d'indice *début*, tandis que la première case libre, c'est-à-dire la case dans laquelle on placera la prochaine donnée enfilée, est d'indice $fin + 1$. Enfiler un nouvel élément x consiste donc à introduire x dans la case $T(fin + 1)$ et à remplacer *fin* par $fin + 1$. Défiler un élément consiste d'autre part à remplacer *début* par $début + 1$, après avoir traité l'élément $T(début)$. En pratique, la dimension de T est une constante *dim* : le nombre d'éléments présents simultanément dans T ne peut donc pas excéder *dim*. Si d'autre part le nombre d'entrées et de sorties est grand devant *dim*, on a intérêt à gérer T de manière cyclique pour éviter un débordement : quand il n'y a plus de place à la fin du tableau pour ajouter un élément, on recommence à remplir T à partir de ses premières cases (dans ce cas, *fin* devient plus petit que *début*).

Pseudo-code

On suppose qu'on utilise un tableau à N cases indicées de 1 à N .

On utilise deux variables *début* et *fin*. On peut envisager un traitement cyclique de la file mais ce n'est pas ce qui est fait ici. On fait en sorte que la file soit vide si et seulement si $fin < début$. La variable *début* est initialisée à 1 et la variable *fin* est initialisée à 0. Quand la file n'est pas vide, les données se trouvent entre l'indice *début* compris et l'indice *fin* compris.

Pour la fonction *enfiler*, on choisit ci-dessous de ne rien faire si on appelle la fonction *enfiler* alors que *fin* vaut N . Dans un code plus élaboré, ce cas devrait être traité.

enfiler(clé)

Si $fin < N$, alors

- $fin \leftarrow fin + 1$;
- $T[fin] \leftarrow clé$;

Pour la fonction *défiler*, on choisit ci-dessous de ne rien faire si on appelle la fonction *défiler* alors que la file est vide. Dans un code plus élaboré, ce cas devrait être traité.

défiler

Si $début \leq fin$, alors

- $clé \leftarrow T[début]$;
- $début \leftarrow début + 1$;
- renvoyer *clé*;

Implémentation d'une file par une liste chaînée

Si la file est représentée par une liste chaînée, il est commode de maintenir à jour deux pointeurs, *début* et *fin*, la liste étant chaînée de *début* vers *fin*. Pour défiler, on décale *début* en le faisant pointer sur le maillon qui suit celui sur lequel il pointe avant cette opération, après avoir traité l'élément du maillon supprimé. Pour enfiler un nouvel élément x , on crée un

nouveau maillon M dans lequel on place x et dans lequel l'adresse du maillon suivant est l'adresse nulle. Puis on ajoute M à la fin de la liste à l'aide du pointeur fin que l'on décale pour le faire pointer sur M .

Pseudo-code

On utilise une variable *début* et une variable *fin*. On fait en sorte que la file soit vide si et seulement si *fin* vaut *rien*. Les variables *début* et *fin* sont initialisées à *rien*. On constatera dans le pseudo-code ci-dessous que *fin* vaut *rien* si et seulement si *début* vaut *rien*. Une fonction nommée *nouvelle_cellule* permet de créer une cellule et retourne l'adresse de celle-ci.

enfiler(clé)

- $p \leftarrow \text{nouvelle_cellule};$
- $p.\text{donnée} \leftarrow \text{clé};$
- $p.\text{suivant} \leftarrow \text{rien};$
- Si $fin \neq \text{rien}$, alors $fin.\text{suivant} \leftarrow p;$
sinon $début \leftarrow p;$
- $fin \leftarrow p;$

Pour la fonction *défiler*, on choisit ci-dessous de ne rien faire si on appelle la fonction *défiler* alors que la file est vide. Dans un code plus élaboré, ce cas devrait être traité.

défiler

Si $début \neq \text{rien}$, alors

- $clé \leftarrow début.donnée;$
- $début \leftarrow début.suivant;$
- si $début = \text{rien}$ alors $fin \leftarrow \text{rien};$
- renvoyer $clé;$

Dans cette dernière fonction, il peut convenir, selon le langage de programmation, de libérer l'espace-mémoire qui était alloué à la cellule contenant la donnée qui a été défilée.

2.3. Arbres

2.3.1 Arbres généraux

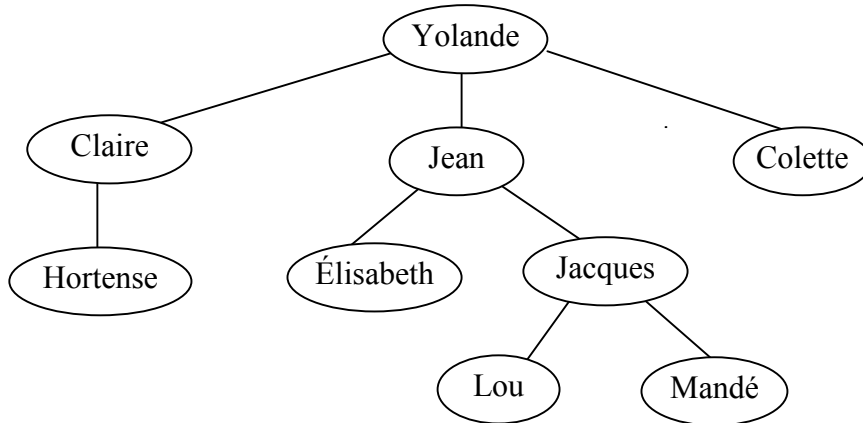
La définition que nous donnons d'un arbre est récursive¹.

Un arbre est une structure composée d'éléments appelés *nœuds* ; il est constitué d'un nœud particulier appelé *racine* et d'une suite ordonnée éventuellement vide $A_1, A_2, \dots, A_p$

¹ Notons que les arbres dont il s'agit ici ne correspondent pas exactement aux arbres de la théorie des graphes, même si certaines propriétés leur sont communes.

d'arbres disjoints appelés *sous-arbres* de la racine. Un arbre contient donc au moins un nœud : sa racine.

Exemple : la figure suivante représente l'arbre d'une descendance ; la racine de cet arbre contient Yolande ; il est constitué de trois sous-arbres ($p = 3$) : A_1 , de racine Claire, contient Claire et Hortense ; A_2 , de racine Jean, contient Jean, Élisabeth, Jacques, Lou et Mandé ; A_3 , de racine Colette, est réduit à Colette. L'arbre est ici dessiné de haut en bas (la racine est en haut) et les sous-arbres de gauche à droite ; d'autres représentations sont possibles.



Plusieurs définitions sont à retenir ; nous illustrons les définitions avec l'exemple ci-dessus en identifiant un nœud avec la chaîne de caractères qu'il contient :

- les fils d'un nœud sont les racines de ses sous-arbres ; sur l'exemple, les fils de Jean sont Élisabeth et Jacques ;
- le père d'un nœud x autre que la racine est l'unique nœud dont x est un fils ; sur l'exemple, Jacques est le père de Lou et de Mandé ; la racine d'un arbre n'a pas de père ;
- un *nœud interne* est un nœud qui a au moins un fils ; sur l'exemple, Claire est un nœud interne ;
- une *feuille* d'un arbre est un nœud sans fils ; sur l'exemple, Mandé est une feuille ;
- les *ancêtres* d'un nœud a sont les nœuds qui sont sur le chemin entre la racine (incluse) et a (inclus) ; les ancêtres de a différents de a sont les ancêtres propres de a ; sur l'exemple, les ancêtres de Jacques sont Jacques, Jean et Yolande ;
- les *descendants* d'un nœud a sont les nœuds qui appartiennent au sous-arbre de racine a ; les descendants de a différents de a sont les descendants propres de a ; sur l'exemple, les descendants de Jean sont Jean, Élisabeth, Jacques, Lou et Mandé.
- la *profondeur* d'un nœud est la longueur du chemin allant de la racine à ce nœud ; la profondeur de la racine est donc nulle et la profondeur d'un nœud autre que la racine vaut un de plus que la profondeur de son père ; la profondeur d'un nœud peut aussi être appelée *niveau* ou *hauteur* du nœud ; sur l'exemple, la profondeur d'Élisabeth vaut 2 ;
- la *hauteur* d'un arbre est la profondeur maximum de ses nœuds ; la hauteur de l'arbre donné en exemple vaut 3.

Pour connaître un arbre, on procède généralement en mémorisant la racine (ou plus précisément son adresse) ; il suffit alors que chaque nœud connaisse ses nœuds fils pour pouvoir retrouver tout l'arbre.

2.3.2 Parcours d'un arbre général

Parcourir un arbre, c'est examiner une fois et une seule chacun de ses nœuds, l'ordre des examens dépendant d'une certaine règle. On distingue les deux types suivants. Diverses opérations peuvent être effectuées quand on examine un nœud (par exemple, lui attribuer un numéro).

Le parcours en préordre

Parcourir un arbre en *préordre*, c'est :

- examiner la racine ;
- parcourir en préordre le premier sous-arbre ;
- ...
- parcourir en préordre le dernier sous-arbre.

Ce parcours est aussi dit *parcours en ordre préfixe*.

On peut se convaincre que la complexité de ce parcours est du même ordre que le nombre de nœuds de l'arbre.

Exemple : le parcours en préordre de l'arbre donné en exemple considère successivement les nœuds Yolande, Claire, Hortense, Jean, Élisabeth, Jacques, Lou, Mandé, Colette.

Le parcours en postordre

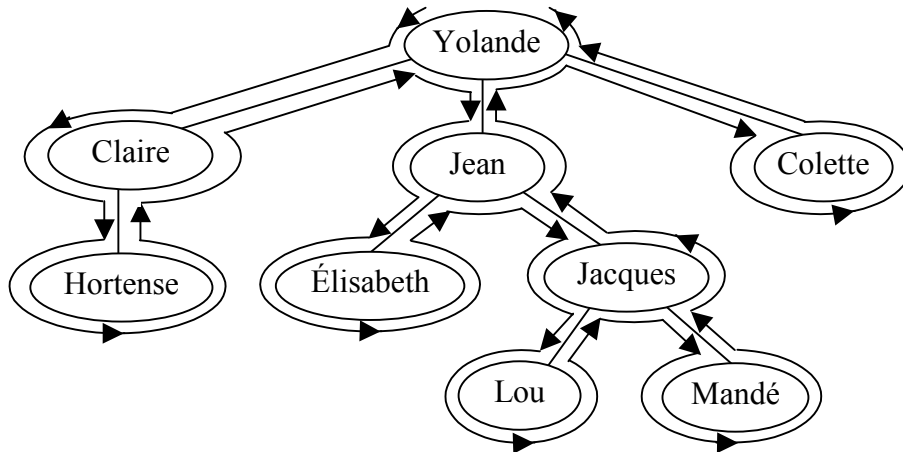
Parcourir un arbre en *postordre*, c'est :

- parcourir en postordre le premier sous-arbre ;
- ...
- parcourir en postordre le dernier sous-arbre ;
- examiner la racine.

Ce parcours est aussi dit *parcours en ordre postfixe* ou *suffixe*. Le parcours en ordre postfixe a clairement la même complexité que le parcours en préordre, c'est-à-dire de l'ordre du nombre de nœuds de l'arbre.

Exemple : le parcours en postordre de l'arbre donné en exemple considère successivement les nœuds Hortense, Claire, Élisabeth, Lou, Mandé, Jacques, Jean, Colette, Yolande.

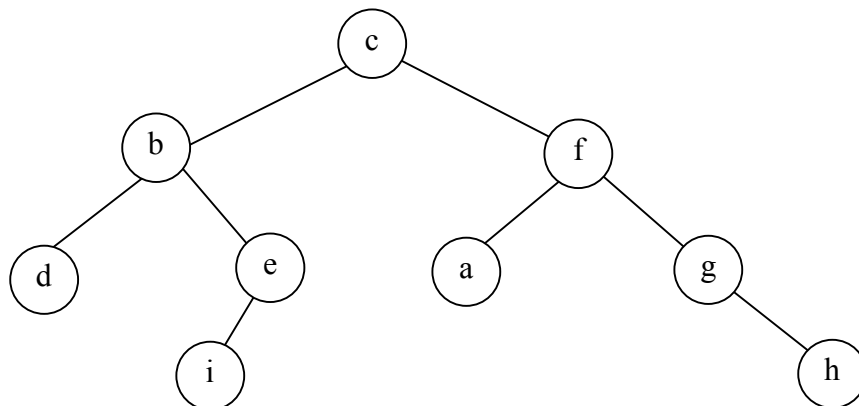
Remarque : on peut voir les ordres obtenus par les parcours en préordre et en postordre sur l'ensemble des nœuds d'une autre façon en considérant ce qu'on appelle « un parcours à main gauche » de l'arbre. On part de la racine et on suit le tracé de l'arbre comme l'indique la figure ci-dessous. On s'aperçoit que l'examen en préordre est obtenu en examinant les nœuds à la première rencontre alors que l'examen en postordre est obtenu en examinant les nœuds à la dernière rencontre.



2.3.3 Arbres binaires

Un *arbre binaire* est soit vide, soit constitué d'un nœud particulier appelé racine et de deux sous-arbres binaires disjoints appelés sous-arbre gauche et sous-arbre droit. On notera donc que, formellement, un arbre binaire n'est pas un arbre, au sens donné ci-dessus (un arbre n'est pas vide et n'utilise pas la notion de droite et de gauche). Cependant, la forte parenté qui existe entre ces deux concepts fait que l'on est souvent amené à étendre aux arbres binaires certaines définitions valables en principe seulement pour les arbres généraux. Ce sera par exemple le cas plus bas pour les parcours en préordre ou en postordre.

Exemple d'arbre binaire :

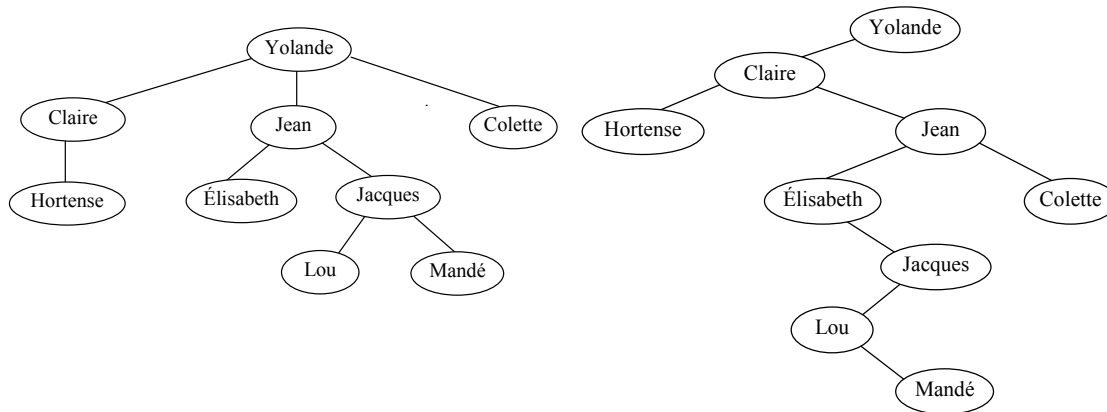


La terminologie est la même que celle des arbres généraux ; néanmoins, si on considère les fils d'un nœud, on parlera, s'ils existent, du fils gauche et du fils droit. Sur l'exemple, on remarque que certains nœuds peuvent avoir seulement un fils droit ou seulement un fils gauche ; par exemple, le nœud e (*i.e.* contenant la donnée e) admet uniquement un fils gauche, le nœud i, et pas de fils droit ; si on considérait l'exemple comme un arbre général, on dirait que e a un seul fils, le nœud i, et le lien entre e et i serait dessiné verticalement.

Pour coder un arbre binaire, on fait souvent correspondre à chaque nœud une structure contenant la donnée et deux adresses, une adresse pour chacun des deux nœuds fils, avec la convention qu'une adresse nulle indique un arbre binaire vide. Il suffit alors de mémoriser l'adresse de la racine pour pouvoir reconstituer tout l'arbre.

Remarque : on code souvent un arbre général en utilisant un arbre binaire ; pour cela on associe à chaque nœud de l'arbre initial un nœud de l'arbre binaire ; le fils gauche d'un nœud

dans l'arbre binaire correspond au premier fils du nœud correspondant de l'arbre initial (on met une adresse nulle s'il n'y a aucun fils) alors que le fils droit de ce nœud correspond au premier « frère cadet », c'est-à-dire au nœud de l'arbre initial qui a même père que le nœud considéré et qui se trouve immédiatement à sa droite (on met une adresse nulle s'il n'y a aucun frère cadet). Ainsi, l'arbre général donné en exemple précédemment et retracé ci-dessous à gauche peut être codé avec l'arbre binaire représenté à droite, en changeant bien entendu l'interprétation des liens.



Parcours ; le parcours en ordre symétrique

Les deux parcours définis pour les arbres généraux sont aussi définis pour les arbres binaires. Pour l'exemple ci-dessus, le parcours en préordre donne l'ordre : c, b, d, e, i, f, a, g, h et le parcours en postordre donne l'ordre : d, i, e, b, a, h, g, f, c.

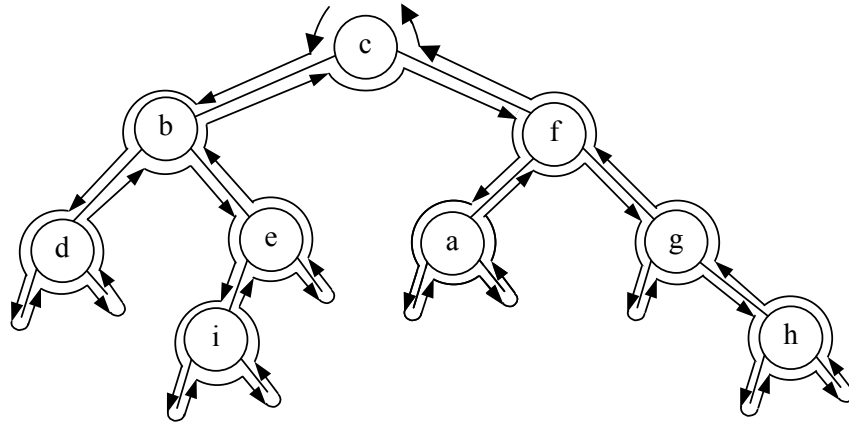
On considère pour les arbres binaires un troisième ordre : l'*ordre symétrique*. Parcourir un arbre binaire en ordre symétrique, c'est :

- si l'arbre binaire est non vide, alors
 - parcourir en ordre symétrique le sous-arbre gauche ;
 - examiner la racine ;
 - parcourir en ordre symétrique le sous-arbre droit.

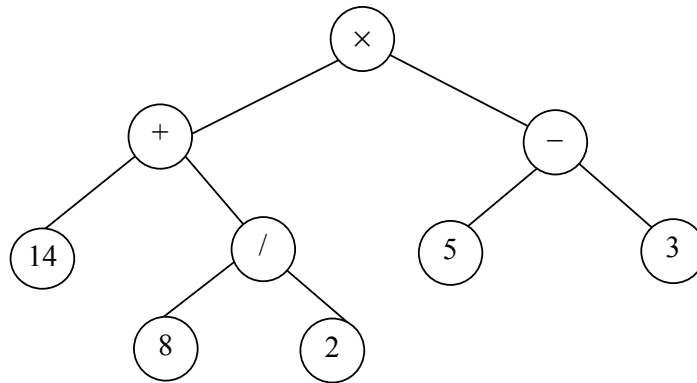
Ce parcours est aussi dit parcours en *ordre infixe* ou *milieu*. La complexité du parcours en ordre symétrique est, de même que pour les autres parcours, de l'ordre du nombre de nœuds de l'arbre.

Exemple : le parcours en ordre symétrique de l'arbre binaire donné en exemple conduit à l'ordre : d, b, i, e, c, a, f, g, h.

Remarque : on peut illustrer les trois parcours des arbres binaires à l'aide du parcours à main gauche ; il est plus agréable pour cela de faire figurer sur le tracé de l'arbre les sous-arbres vides comme il est fait ci-dessous ; le préordre correspond à la première fois que l'on rencontre les nœuds, l'ordre symétrique à la deuxième fois et le postordre à la troisième fois.



Exemple d'application : on peut utiliser un arbre binaire pour représenter une expression arithmétique composée à partir des entiers et des quatre opérateurs binaires : $+$, $-$, $\times$, $/$. L'arbre ci-dessous représente l'expression : $(14 + (8 / 2)) \times (5 - 3)$



Par construction, dans cet arbre, tout nœud interne a un fils gauche et un fils droit.

Les trois parcours de cet arbre donnent :

- parcours en préordre : $\times + 14 / 8 2 - 5 3$; on reconnaît la forme polonaise de l'expression ;
- parcours en ordre infixe : $14 + 8 / 2 \times 5 - 3$; il faudrait ajouter les parenthèses pour retrouver l'expression initiale ;
- parcours en postordre : $14 8 2 / + 5 3 - \times$; on reconnaît la forme polonaise inverse de l'expression.

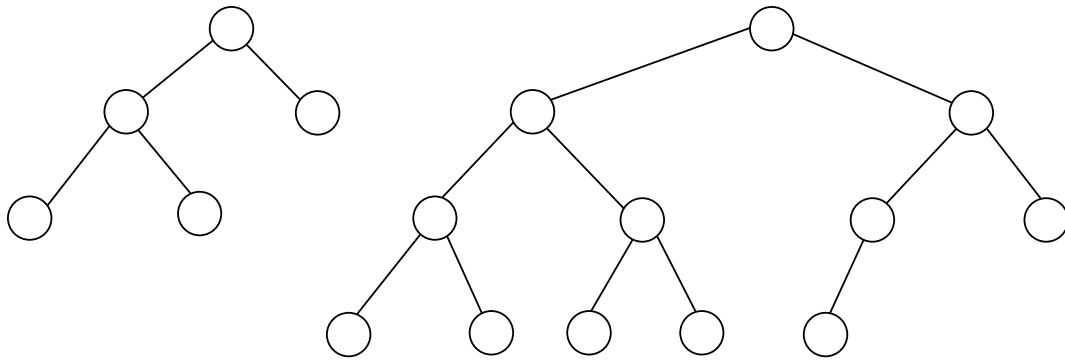
Quelques arbres binaires particuliers

Un arbre binaire est dit *complet* si tout nœud interne a exactement deux fils².

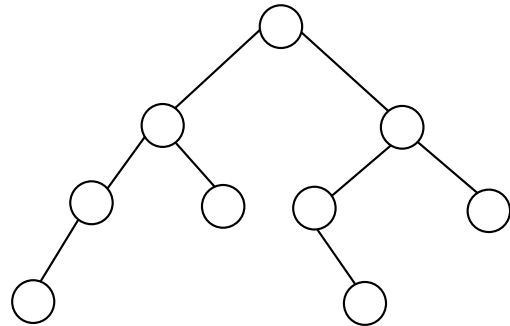
Un arbre binaire est dit *parfait* si, en appelant h la hauteur de l'arbre, les niveaux de profondeur $0, 1, \dots, h - 1$ sont complètement remplis alors que le niveau de profondeur h est rempli en partant de la gauche ; la structure d'un arbre binaire parfait est complètement

² Certains auteurs adoptent une définition plus contraignante pour les arbres complets, en imposant de plus à toutes les feuilles d'un tel arbre d'avoir la même profondeur.

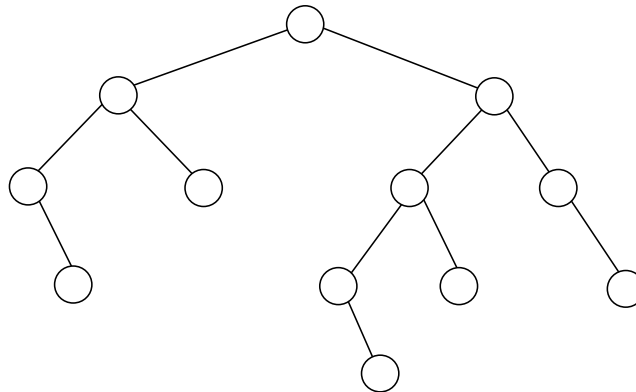
déterminée par son nombre de nœuds ; nous donnons ci-après les arbres binaires parfaits à 5 nœuds et à 12 nœuds. On rencontre aussi quelquefois le qualificatif de *presque-complet* pour un tel arbre.



En revanche, l'arbre ci-contre n'est pas parfait.



Un arbre binaire est dit *équilibré* si, pour tout nœud de l'arbre, les sous-arbres gauche et droit ont des hauteurs qui diffèrent au plus de 1. L'arbre ci-dessous est équilibré.



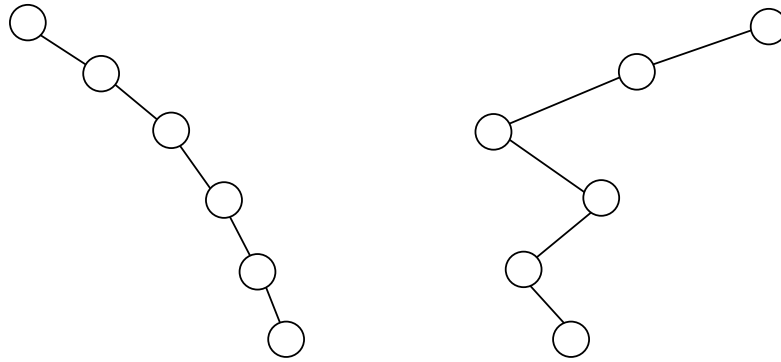
Un arbre binaire parfait est équilibré.

On pourrait démontrer que la hauteur h d'un arbre binaire équilibré ayant n nœuds vérifie :

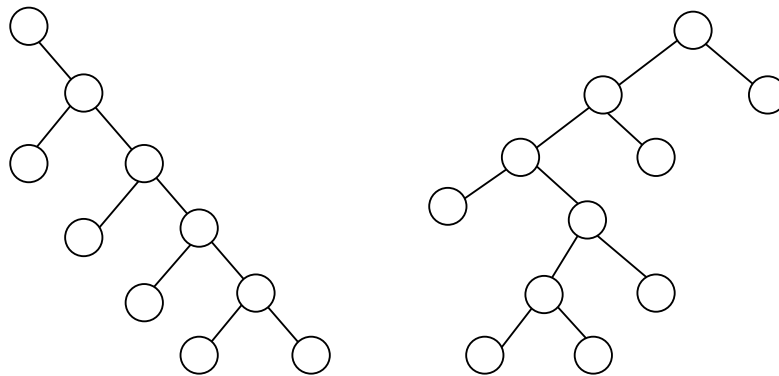
$$\log_2(n + 1) \leq h \leq 1,44 \log_2(n + 2),$$

d'où on peut déduire facilement que la profondeur moyenne des nœuds d'un arbre binaire équilibré est du même ordre que $\ln n$.

En ce qui concerne un arbre binaire quelconque de n nœuds, on établirait facilement que la profondeur ainsi que la hauteur moyenne des feuilles ont toujours un ordre de grandeur compris entre $\ln n$ et n . La hauteur de l'ordre de n est atteinte pour des arbres qui sont « tout en hauteur » comme on peut en voir ci-après ; ces arbres seront dits *dégénérés*.



Des arbres binaires complets peuvent aussi avoir une hauteur de l'ordre du nombre de nœuds comme l'illustrent les deux arbres ci-dessous, qualifiés aussi de *dégénérés*.



3. Recherche et tri

3.1. Introduction

On considère des données appartenant à un ensemble totalement ordonné. Les trier consiste alors à les ranger en ordre croissant ou décroissant. Quant à rechercher une donnée, c'est soit simplement déterminer si la donnée figure ou non, soit, dans le cas où la donnée figure, trouver l'endroit où cette donnée est stockée.

Les opérations de recherche et de tri consomment un pourcentage important du temps total de calcul des ordinateurs. C'est pourquoi les gains sur la complexité des algorithmes de tri et de recherche ont une importance considérable : aussi ont-ils été très étudiés et perfectionnés.

Dans tout ce chapitre, les données que nous considérerons seront des entiers. Néanmoins, remarquons qu'il est rare de trier simplement des entiers ; on rencontre plus fréquemment des tris de structures selon un champ de cette structure. On peut trier des structures comportant un nom d'élève et une note de cet élève par liste alphabétique des noms des élèves ou par notes croissantes ; on peut trier des factures par dates croissantes ; on peut trier des cartes à jouer par couleurs et à l'intérieur de chaque couleur par niveaux et ainsi de suite. Il faudra considérer que le type des données considérées est un type quelconque sur lequel est définie une relation d'ordre total notée avec les symboles habituels $<$, $>$, $\leq$, $\geq$.

Nous ne traitons pas tous les algorithmes de tri et de recherche dans ce chapitre, ni dans l'ensemble du polycopié, mais seulement une partie de ce qui peut être dit sur le sujet. Nous laissons pour les chapitres suivants les algorithmes de recherche et de tri utilisant une structure d'arbre binaire de recherche ainsi que l'algorithme de tri utilisant une structure de tas.

3.2. Recherche dichotomique dans un tableau trié

On suppose ici que l'on dispose d'un tableau trié par ordre croissant et qu'on souhaite savoir si un élément fixé (i.e. une donnée) figure ou non dans la liste.

On peut bien sûr utiliser une recherche séquentielle qui consiste à regarder successivement, en partant de la gauche du tableau, tous les éléments, selon l'exemple vu en début de ce polycopié. Néanmoins, il sera plus astucieux d'utiliser une *recherche dichotomique*.

La recherche dichotomique consiste à comparer l'élément, ou un des deux éléments, qui se trouve au centre de la liste avec l'élément recherché ; si cet élément central est égal à l'élément recherché, c'est terminé ; s'il est supérieur à l'élément recherché, on abandonne la moitié droite de la liste (correspondant aux éléments plus grands que l'élément central) et on recommence avec la liste formée par la moitié gauche (correspondant aux éléments plus petits que l'élément central) de la liste initiale ; s'il est inférieur à l'élément recherché, on abandonne la moitié gauche de la liste (correspondant aux éléments plus petits que l'élément central) et on recommence avec la liste formée par la moitié droite (correspondant aux éléments plus grands que l'élément central) de la liste initiale. Et ainsi de suite jusqu'à ce que la liste restante soit vide (recherche infructueuse) ou bien avoir trouvé l'élément cherché (recherche fructueuse).

Écrivons le pseudo-code de cet algorithme qui renvoie *vrai* ou *faux* selon que l'élément recherché est trouvé ou non. Si on recherche l'indice de l'élément dans le cas où il figure, il faut adapter l'algorithme.

Recherche dichotomique dans un tableau trié

Les données sont rangées dans un tableau T entre les indices 1 et n . On utilise trois variables entières notées *gauche*, *droite* et *milieu*. On utilise aussi une variable booléenne notée *trouvé*. L'élément recherché s'appelle *clé*. La relation d'ordre entre les éléments s'exprime avec le signe ' $<$ '. La division utilisée est la division entière.

- $gauche \leftarrow 1$;
- $droite \leftarrow n$;
- $trouvé \leftarrow faux$;
- tant que $((non\ trouvé) \text{ et } (gauche \leq droite))$
 - $milieu \leftarrow (gauche + droite)/2$;
 - si $(clé < T[milieu])$, alors $droite \leftarrow milieu - 1$;
 - sinon si $(clé > T[milieu])$, alors $gauche \leftarrow milieu + 1$;
 - sinon $trouvé \leftarrow vrai$;
- renvoyer *trouvé*.

Nous allons établir la complexité de cet algorithme. Pour chaque passage dans la boucle tant que, on fait un nombre d'opérations élémentaires majoré par une constante ; par ailleurs, à chaque passage dans la boucle, la longueur de la liste est divisée par 2 (même un peu plus), ce qui fait que si on note p le nombre total de passages dans la boucle, au bout de p passages, la longueur de la liste est majorée par $n / 2^p$, et au moins égale à 1 : $n / 2^p \geq 1$, et donc : $2^p \leq n$; d'où $p \leq \log_2(n)$. La complexité est au pire de l'ordre de $\log_2(n)$ ou, ce qui revient au même, de $\ln(n)$.

3.3. Complexité d'un algorithme de tri

Comme pour tout algorithme, la complexité d'un algorithme de recherche ou de tri peut s'évaluer soit dans le « pire des cas », c'est-à-dire pour un ordre initial des données spécialement défavorable pour cet algorithme, soit dans le « meilleur des cas », soit « en

moyenne ». Par « en moyenne », on entend ici la moyenne arithmétique des complexités sur toutes les permutations possibles des données, que l'on suppose, pour ce calcul, deux à deux distinctes.

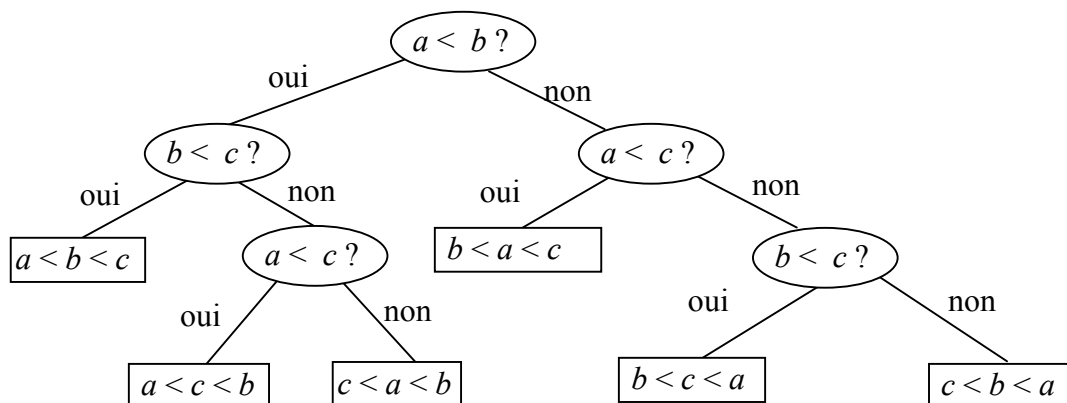
Si un algorithme de tri doit souvent être exécuté mais qu'il n'est pas impératif d'avoir un temps d'exécution maximum très petit, on pourra utiliser un algorithme de bonne complexité en moyenne mais dont la complexité au pire peut être médiocre. C'est le cas d'un certain nombre de transactions que l'on peut exécuter en temps différé ; le fait que, dans le « pire des cas », de tels algorithmes ne soient pas très performants n'est pas pénalisant, le pire étant peu probable en principe. En revanche, lorsqu'on travaille en temps réel, on s'intéresse souvent à améliorer la complexité dans le pire des cas. En effet, même si elles sont rares, des attentes très longues peuvent être inacceptables.

La plupart des algorithmes de tri sont fondés sur des comparaisons successives entre les données pour déterminer la permutation correspondant à l'ordre croissant des données. Nous appellerons *tri comparatif* un tel tri. La complexité de l'algorithme a alors le même ordre de grandeur que le nombre de comparaisons entre les données faites par l'algorithme. Nous ne verrons dans ce polycopié que des tris comparatifs.

Signalons néanmoins le *tri baquet* ou le *tri par paquets*, qui sont deux algorithmes de tri non comparatifs. Une forme dégénérée du tri baquet suit le principe ci-dessous. Supposons qu'il s'agisse de trier des entiers compris entre 0 et N ; on utilise un tableau supplémentaire T (autre que celui qui contient éventuellement les données) indicé de 0 à N ; la case d'indice p de T contiendra après l'algorithme le nombre de fois que l'entier p figure dans la liste des entiers. On initialise à 0 toutes les cases du tableau T ; on considère successivement tous les entiers de la liste à trier ; si l'entier considéré vaut p , on incrémente $T[p]$ de 1 ; après avoir traité toutes les données, il suffit de relire le tableau T en partant de l'indice 0 pour connaître la liste triée (si par exemple $T[0]$ vaut 1, $T[1]$ et $T[2]$ valent 0, $T[3]$ vaut 2 et $T[4]$ vaut 1, la liste triée commence par 0, 3, 3, 4). Dans ce tri, on ne compare jamais les données entre elles. En notant n le nombre de données triées, ce tri a pour complexité $n + N$ ou, ce qui revient au même, $\max(n, N)$.

De façon à évaluer la complexité théorique des tris comparatifs, nous allons nous intéresser au nombre de comparaisons faites par un tel algorithme de tri. Nous allons employer une approche intuitive.

Considérons un algorithme de tri de n données ; la réponse cherchée par le tri est l'une des $n!$ permutations possibles des données. Prenons un exemple ; on considère un algorithme pour trier trois données a , b et c décrit par l'arbre ci-dessous ; cet algorithme correspond au fonctionnement du tri insertion que nous verrons plus loin :



Cet arbre signifie : la première comparaison faite est « $a < b$? ». Si la réponse est oui, la comparaison suivante est « $b < c$? », si la réponse est non c'est « $a < c$? ». Lorsqu'une permutation est déterminée, on est dans une feuille de l'arbre. On voit qu'on peut avoir plus ou moins de chance ; pour deux des permutations, on fait deux comparaisons, pour les quatre autres, on fait trois comparaisons. Le plus grand nombre de comparaisons est 3, le meilleur est 2 et le nombre moyen de comparaisons est : $(2 \times 2 + 4 \times 3) / 6 \approx 2,67$.

Pour tout algorithme de tri comparatif, la première comparaison faite consiste à prendre deux des données a et b de la liste et à poser la question : « a-t-on $a < b$? », ce qui divise les $n!$ permutations envisageables en deux parties égales ; les comparaisons suivantes ne garderont pas une telle symétrie : on aura de la chance quand il y aura moins de la moitié des permutations restant possibles qui correspondent à la réponse obtenue et de la malchance sinon. Si on s'intéresse à la complexité dans le pire des cas d'un algorithme, il faut supposer que la permutation considérée correspond toujours à un cas où on n'a jamais de chance, c'est-à-dire où chaque comparaison élimine au plus la moitié des permutations encore envisageables. Dans ce cas, après k comparaisons, il restera au moins $n! / 2^k$ permutations envisageables ; il faut avoir $n! / 2^k < 2$ pour avoir une permutation unique ; donc on aura effectué au moins de l'ordre de $\log_2(n!)$ comparaisons. Comme $\log_2(n!)$ est équivalent à $n \log_2(n)$, on a :

Résultat : tout algorithme comparatif fait dans le pire des cas au moins de l'ordre de $n \ln(n)$ comparaisons.

En ce qui concerne le nombre moyen de comparaisons, nous admettons que la hauteur moyenne des feuilles d'un arbre binaire ayant p feuilles est au moins égale à $\log_2(p)$. Le nombre moyen de comparaisons d'un algorithme de tri est la hauteur moyenne des feuilles de l'arbre tracé de la même façon que sur notre exemple. Comme $\log_2(n!)$ est équivalent à $n \log_2(n)$, on a :

Résultat : tout algorithme comparatif fait en moyenne au moins de l'ordre de $n \ln(n)$ comparaisons.

Nous allons voir plus loin le *tri sélection* et le *tri insertion* de complexité en $O(n^2)$, qui seront simples à écrire mais peu efficaces, ; en effet, nous verrons aussi le *tri rapide* et le *tri par arbre binaire de recherche* qui sont en moyenne en $O(n \ln n)$, mais dans le pire des cas en $O(n^2)$; nous verrons enfin le *tri tas* qui est dans le pire des cas en $O(n \ln n)$.

3.4. Tri sélection

Le tri sélection consiste à chercher la plus petite donnée de la liste, à mettre celle-ci en première position, puis à chercher la plus petite donnée parmi les données autres que la première, la mettre en deuxième position, et ainsi de suite.

Dans le pseudo-code donné ci-dessous, on suppose qu'on dispose d'une fonction *échanger* telle que, si T est un tableau et i et j deux indices de ce tableau, *échanger*(T, i, j) échange les données du tableau T qui se trouvent aux indices i et j .

Tri sélection

Les données sont rangées dans un tableau T entre les indices 1 et n .

On utilise trois variables entières notées i , j et $indicePetit$, et une variable min du même type que les données de la liste. Pour chaque itération de la boucle portant sur i , on cherche le plus petit élément de la liste qui se trouve entre les indices i et n .

```

➤ pour  $i$  qui varie de 1 à  $n - 1$  ;
  o    $indicePetit \leftarrow i$  ;
  o    $min \leftarrow T[i]$  ;
  o   pour  $j$  qui varie de  $i + 1$  à  $n$ , faire
    • si  $T[j] < min$ 
      ▪  $indicePetit \leftarrow j$  ;
      ▪  $min \leftarrow T[j]$  ;
  o    $échanger(T, i, indicePetit)$ .

```

La complexité de ce tri est aussi bien dans le pire des cas que dans le meilleur des cas (et donc qu'en moyenne) en $O(n^2)$. Ce n'est donc pas un tri très efficace, mais en revanche c'est un tri très simple.

3.5. Tri insertion

Le principe du tri par insertion est le suivant. On souhaite trier une liste de n données ; on procède par étape ; à la i^{e} étape (i variant de 1 à $n - 1$), on suppose que les i premières données sont déjà triées ; on considère alors la $(i + 1)^{\text{e}}$ donnée que l'on appelle *clé* ; on la compare successivement aux données précédentes, en commençant par la i^{e} puis en remontant dans la liste jusqu'à trouver la bonne place de *clé* (c'est-à-dire entre deux données successives, l'une étant plus petite et l'autre plus grande que *clé*), ou bien en tout premier si *clé* est plus petite que les i premières données ; au fur et à mesure, on décale « d'une case vers la droite » les données plus grandes que *clé* de façon à anticiper la place de ces données après insertion de *clé* ; on met *clé* à la bonne place ; à l'issue de cette étape, les $i + 1$ premières données sont donc triées.

Tri insertion

Les données sont rangées dans un tableau T entre les indices 1 et n .

On utilise deux variables entières notées i, j et une variable *clé* du même type que les données de la liste.

```

➤ pour  $i$  qui varie de 2 à  $n$ 
  o    $j \leftarrow i$  ;
  o    $clé \leftarrow T[j]$  ;
  o   tant que  $j \geq 2$  et  $T[j - 1] > clé$ , faire
    •  $T[j] \leftarrow T[j - 1]$  ;
    •  $j \leftarrow j - 1$  ;
  o    $T[j] \leftarrow clé$ .

```

Ce tri est en $O(n^2)$ dans le pire des cas (cas où la liste est triée dans l'ordre inverse) et en $O(n)$ dans le meilleur des cas (cas où la liste serait déjà triée avant application de l'algorithme) ; si on considère que toutes les permutations des données sont équiprobables, on

vérifie facilement que le tri est en moyenne en $O(n^2)$. C'est donc encore un tri peu efficace mais qui a aussi le mérite de la simplicité.

Le tri insertion a une complexité en moyenne du même ordre que celle du tri sélection ; néanmoins, si les données à trier sont souvent presque triées (dans le sens souhaité), il vaudra mieux choisir le tri insertion.

3.6. Tri rapide

Le principe du tri rapide est le suivant. On utilise une fonction nommée *partition* qui s'applique à une liste ; cette fonction extrait une donnée, nommée *clé*, de la liste, par exemple la première (c'est ce que nous choisirons plus bas) puis elle réorganise la liste de sorte qu'on trouve d'abord les données ne dépassant pas la valeur de *clé* (on appellera sous-liste gauche cette partie de la liste), puis la donnée *clé*, puis les données supérieures à *clé* (on appellera sous-liste droite cette partie de la liste). Le tri rapide peut être défini récursivement. Pour appliquer le tri rapide à une liste ayant au moins deux données, on commence par appliquer la fonction *partition*, puis on applique le tri rapide à la sous-liste gauche, puis on applique le tri rapide à la sous-liste droite.

La fonction *partition* peut être implémentée de différentes façons ; il importe simplement que sa complexité pour une liste de longueur n soit en $O(n)$, pour que le tri rapide ait une bonne complexité (voir plus bas). Nous supposons que les données sont dans un tableau T et que ce tableau contient une case supplémentaire à la droite des données, qui contient une donnée strictement supérieure à toutes les données de la liste ; cette condition est nécessaire pour que, dans le pseudo-code donné ci-dessous, la variable i ne risque pas de croître « indéfiniment » ; il ne faudra pas l'oublier au moment d'une programmation (ou bien il faudra modifier légèrement le pseudo-code ci-dessous pour éviter le débordement sur la droite du tableau). Dans le pseudo-code suivant, *partition* renvoie l'indice du tableau où *clé* est rangée. La procédure *échanger* est définie comme plus haut.

Fonction *partition*(*g, d*)

On s'intéresse aux données qui sont dans le tableau *T* entre les indices *g* et *d* inclus.

On utilise deux variables entières notées *i, j* et une variable *clé* du même type que les données de la liste.

- $clé \leftarrow T[g]$;
- $i \leftarrow g + 1$;
- $j \leftarrow d$;
- tant que $i \leq j$, faire
 - tant que $T[i] \leq clé$, faire $i \leftarrow i + 1$;
 - tant que $T[j] > clé$, faire $j \leftarrow j - 1$;
 - si $i < j$, alors
 - $échanger(T, i, j)$;
 - $i \leftarrow i + 1$;
 - $j \leftarrow j - 1$;
- $échanger(T, g, j)$;
- renvoyer *j*.

Prenons un exemple. Les données sont les valeurs qui figurent dans la seconde ligne du tableau *T* suivant, la première précisant les indices des cases de *T* :

1	2	3	4	5	6	7	8	9	10	11	12
9	4	18	11	15	5	17	1	10	7	12	6

On suppose qu'on applique la fonction partition avec $g = 1$ et $d = 12$. La variable *clé* vaut $T[1] = 9$. Au premier passage dans la boucle externe, *i* s'arrête à 3, *j* s'arrête à 12. On procède à l'échange et on obtient :

1	2	3	4	5	6	7	8	9	10	11	12
9	4	6	11	15	5	17	1	10	7	12	18

Après quoi *i* passe 4 et *j* à 11.

Au deuxième passage dans la boucle, *i* s'arrête à 4 et *j* s'arrête à 10. On procède à l'échange et on obtient :

1	2	3	4	5	6	7	8	9	10	11	12
9	4	6	7	15	5	17	1	10	11	12	18

avec $i = 5$ et $j = 9$.

Au troisième passage dans la boucle, *i* s'arrête à 5 et *j* s'arrête à 8. On procède à l'échange et on obtient :

1	2	3	4	5	6	7	8	9	10	11	12
9	4	6	7	1	5	17	15	10	11	12	18

avec $i = 6$ et $j = 7$.

Au quatrième passage dans la boucle, i s'arrête à 7 et j s'arrête à 6. Le test « $i < j$? » est négatif car on a $i \geq j$. On sort de la boucle externe « tant que $i \leq j$ » car on a $i > j$; on échange $T[1]$ avec $T[j]$, i.e. $T[6]$; on obtient :

1	2	3	4	5	6	7	8	9	10	11	12
5	4	6	7	1	9	17	15	10	11	12	18

On retourne l'indice 6. On remarque que la donnée de valeur 9 est à sa place définitive, qu'avant elle se trouvent des données inférieures à 9 et, après, des données supérieures à 9.

On peut maintenant écrire le pseudo-code du tri rapide pour un tableau compris entre les indices g et d . On appellera $tri_rapide(1, n)$ pour trier des données se trouvant entre les indices 1 et n d'un tableau.

Procédure $tri_rapide(g, d)$

On utilise une variable entière notée j .

➤ si $g < d$, alors

- $j \leftarrow partition(g, d)$;
- $tri_rapide(g, j - 1)$;
- $tri_rapide(j + 1, d)$.

Sur notre exemple, il reste à appliquer le tri rapide entre les indices 1 et 5 puis entre les indices 7 et 12.

L'application de $tri_rapide(1, 5)$ appelle $partition(1, 5)$ qui conduit au tableau :

1	2	3	4	5	6	7	8	9	10	11	12
1	4	5	7	6	9	17	15	10	11	12	18

et renvoie la valeur 3. On applique alors $tri_rapide(1, 2)$, qui appelle $partition(1, 2)$ qui ne change pas le tableau et retourne la valeur 1, puis $tri_rapide(1, 0)$ qui ne fait rien, puis $tri_rapide(4, 5)$ qui appelle $partition(4, 5)$ qui conduit au tableau :

1	2	3	4	5	6	7	8	9	10	11	12
1	4	5	6	7	9	17	15	10	11	12	18

et retourne l'indice 4. Puis, $tri_rapide(4, 3)$ ne fait rien ainsi que $tri_rapide(5, 5)$. Le tri rapide entre les indices 1 et 5 est terminé.

L'application de $tri_rapide(7, 12)$ fait appel à $partition(7, 12)$ qui conduit au tableau :

1	2	3	4	5	6	7	8	9	10	11	12
1	4	5	6	7	9	12	15	10	11	17	18

et retourne l'indice 11. Alors, $tri_rapide(7, 10)$ appelle $partition(7, 10)$ qui conduit à :

1	2	3	4	5	6	7	8	9	10	11	12
1	4	5	6	7	9	10	11	12	15	17	18

et retourne l'indice 9. Puis, *tri_rapide*(7, 8) fait appel à *partition*(7, 8) qui ne change pas le tableau et retourne l'indice 7, puis *tri_rapide*(7, 6) ainsi que *tri_rapide*(8, 8) ne font rien. On « remonte » avec *tri_rapide*(12, 12) qui ne fait rien. L'algorithme est terminé.

On remarque que la complexité de la fonction *partition* est bien linéaire en la longueur de la liste considérée.

Le « gros » du travail consiste en l'application des fonctions *partition*.

Le pire des cas pour cet algorithme est celui où la clé vient toujours à une extrémité du sous-tableau allant de l'indice g à l'indice d pour chaque appel à la fonction *partition*. Ce sera par exemple le cas si on trie un tableau déjà trié. Il sera alors fait appel à la fonction *partition* pour des listes de longueur n , puis $n - 1$, puis $n - 2$, ..., puis 2 ; la somme des complexités de ces fonctions donne un algorithme en $O(n^2)$.

Le meilleur des cas est celui où la liste est toujours partagée en deux sous-listes de même longueur. On aura en tout :

- un appel à la fonction *partition* sur une liste de longueur n ,
- deux appels à la fonction *partition* sur des listes de longueurs environ $n/2$,
- quatre appels à la fonction *partition* sur des listes de longueurs environ $n/4$,
- ...

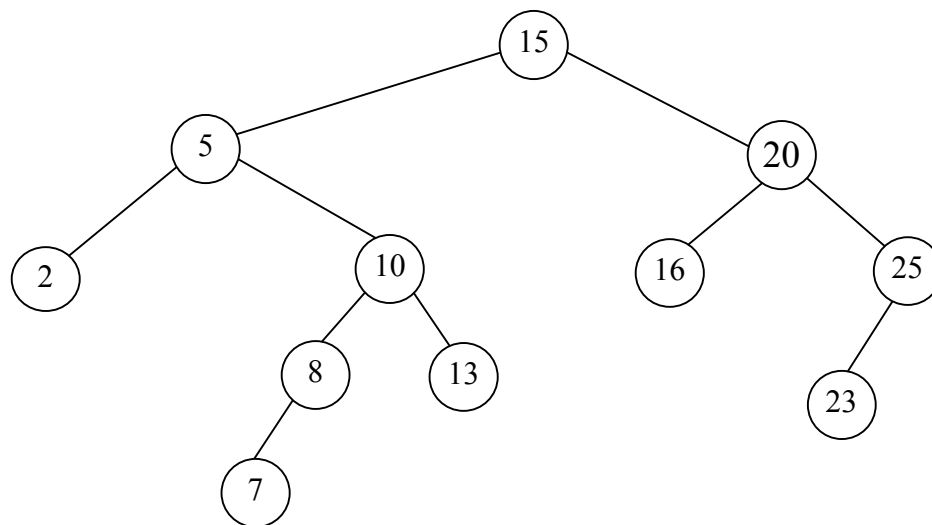
D'où une complexité de l'ordre de n pour la liste de longueur n , une complexité totale de l'ordre de n pour les deux sous-listes de longueur $n/2$, une complexité totale de l'ordre de n pour les quatre sous-listes de longueur $n/4$, ... Comme la longueur est divisée par deux quand on passe de n à $n/2$, puis de $n/2$ à $n/4$, on a en tout une complexité de l'ordre de $n \times \ln n$.

On peut montrer sans trop de difficulté, par une formule de récurrence, que la complexité en moyenne est aussi de l'ordre de $n \times \ln n$; nous ne prouvons pas ici ce résultat.

Remarquons que pour éviter d'avoir une complexité en n^2 lorsqu'on trie une liste déjà triée, on prend souvent, au tout début de la fonction *partition*, une donnée au hasard dans la liste traitée et on l'échange avec la donnée qui se trouve en première position, puis on continue de la façon indiquée précédemment.

3.7. Arbre binaire de recherche

Un *arbre binaire de recherche* est un arbre dont les nœuds appartiennent à un ensemble totalement ordonné et tel que, pour **tout** nœud interne a , les données contenues dans le sous-arbre gauche de a sont inférieures ou égales à la donnée contenue dans le nœud a , qui est elle-même inférieure ou égale aux données contenues dans le sous-arbre droit de a .

Exemple

Par définition d'un arbre binaire de recherche, un parcours en ordre symétrique de l'arbre donne la liste triée des données que contient l'arbre.

Ci-dessous, on supposera que la racine de l'arbre ainsi que les fils gauche et droit d'un nœud sont des adresses de nœuds. Si on a l'adresse *adr* d'un nœud, *adr.donnée* sera la donnée contenue par le nœud (cela serait noté *adr*→*donnée* s'il s'agissait d'un codage en langage C), *adr.fils_gauche* sera l'adresse du fils gauche et *adr.fils_droit* sera l'adresse du fils droit.

Pour insérer une donnée nommée *clé* dans un arbre binaire de recherche, on compare *clé* à la donnée contenue dans la racine ; si *clé* est plus petite, on l'insère dans le sous-arbre gauche ; si elle est plus grande, on l'insère dans le sous-arbre droit. Ceci est décrit plus précisément par la fonction récursive *insérer* dont on donne ci-dessous le pseudo-code. Cette fonction retourne l'adresse de la racine de l'arbre obtenu après insertion dans un arbre binaire de recherche dont la racine a pour adresse *racine* d'un nouveau nœud contenant la donnée *clé*.

Fonction *insérer*(*racine*, *clé*)

- Si *racine* vaut *adresse_nulle*, faire
 - créer un nouveau nœud nommé d'adresse nommée *nouveau* ;
 - *nouveau.donnée* ← *clé* ;
 - *nouveau.fils_gauche* ← *adresse_nulle* ;
 - *nouveau.fils_droit* ← *adresse_nulle* ;
 - retourner *nouveau* ;
- sinon
 - si $clé \leq racine.donnée$, faire
 - *racine.fils_gauche* ← *insérer*(*racine.fils_gauche*, *clé*) ;
 - sinon faire
 - *racine.fils_droit* ← *insérer*(*racine.fils_droit*, *clé*). ;
 - retourner *racine*.

On pourrait écrire d'une manière analogue un algorithme de *recherche* qui retourne vrai ou faux selon qu'une clé reçue en paramètre appartient ou non à l'arbre binaire de recherche.

La complexité d'une insertion ou d'une recherche dans un arbre binaire de recherche est :

- dans le pire des cas égale à la profondeur de l'arbre binaire dans lequel se fait l'insertion ;
- en moyenne égale à la profondeur moyenne des feuilles de l'arbre binaire.

Considérons un arbre binaire de recherche obtenu par insertions successives de n données formant une permutation quelconque, toutes les permutations étant équiprobables. On pourrait montrer qu'en moyenne la hauteur ainsi que la profondeur moyenne des feuilles de cet arbre sont de l'ordre de $\ln(n)$; le pire néanmoins correspond à un arbre dégénéré, où on obtient une hauteur de l'arbre et une profondeur moyenne des feuilles de l'ordre de n .

La recherche d'une donnée dans un arbre binaire de recherche contenant n données est donc au pire en $O(n)$ et en moyenne (sur les arbres binaires possibles et sur les données de ces arbres) en $O(\ln n)$.

Si on veut trier n des données en utilisant un arbre binaire de recherche, il faut :

- construire l'arbre binaire de recherche en partant de l'arbre vide et en insérant les données les unes après les autres ; la complexité pour l'ensemble des insertions est de l'ordre de la somme des profondeurs des nœuds de l'arbre obtenu ; celle-ci a un ordre de grandeur compris entre $n \ln n$ et n^2 , et est en moyenne sur les jeux de données possibles de l'ordre de $n \ln n$;
- on lit en ordre symétrique l'arbre binaire obtenu, opération dont la complexité est du même ordre que le nombre de nœuds de l'arbre, et donc inférieure à celle de la construction de l'arbre.

Le tri par arbre binaire de recherche est donc au pire en $O(n^2)$ et en moyenne en $O(n \ln n)$.

La somme des profondeurs des nœuds d'un arbre binaire de recherche équilibré est de l'ordre de $n \ln n$; on peut écrire un algorithme d'insertion qui, partant d'un arbre binaire de recherche équilibré, fait l'insertion comme indiqué ci-dessus suivi d'une transformation à temps constant qui transforme l'arbre binaire de recherche obtenu en un arbre binaire de recherche équilibré. Nous ne verrons pas ici cette technique. Néanmoins, cette amélioration permet d'obtenir un tri en $O(n \ln n)$ dans le pire des cas et de construire un arbre binaire dans lequel la recherche se fait certainement en temps logarithmique.

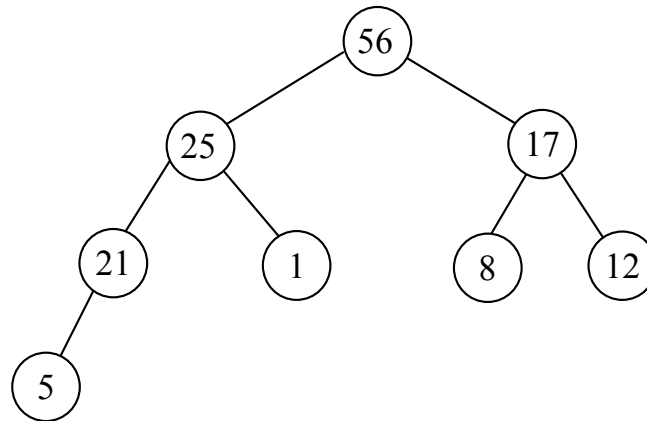
Exercice : comment peut-on supprimer une donnée dans un arbre binaire de recherche ? On s'aidera d'exemples pour concevoir le principe d'un algorithme.

3.8. Structure de tas et tri tas

3.8.1 Définition

On appelle *tas* un arbre binaire parfait, c'est-à-dire où tous les niveaux sont remplis sauf éventuellement le dernier, celui-ci étant rempli de la gauche vers la droite, et dans les sommets duquel figurent des éléments d'un ensemble totalement ordonné, l'élément stocké en un nœud quelconque étant plus grand que les éléments stockés dans ses deux nœuds fils s'il a deux fils, dans son fils s'il a un seul fils. Il n'y a pas d'autre relation d'ordre : un « neveu » peut être plus grand que son « oncle ».

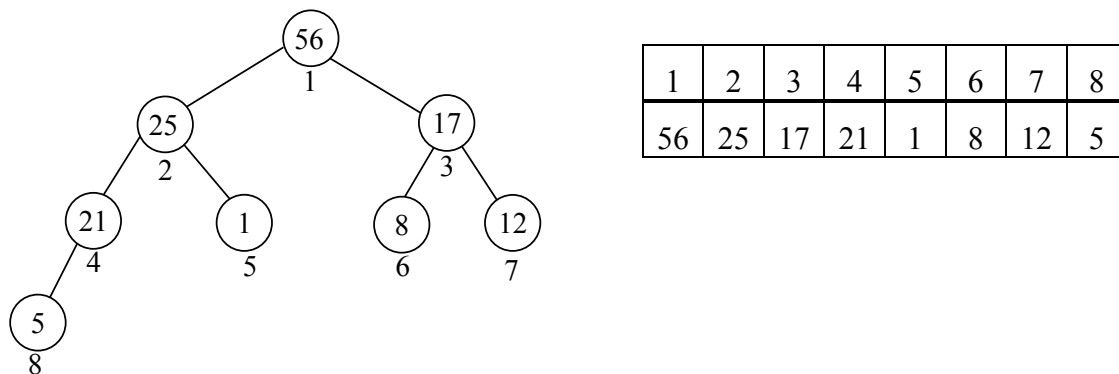
Exemple : ci-dessous est représenté un tas.



3.8.2 Intérêt de la structure d'arbre parfait

Il existe une numérotation canonique des nœuds d'un tas. Si on numérote les nœuds ligne par ligne et de gauche à droite, en donnant le numéro 1 à la racine, on vérifie par récurrence que les numéros des fils du nœud de numéro i , s'ils existent, sont les nœuds de numéros $2i$ et $2i + 1$, de sorte que le numéro du père d'un nœud de numéro i ($i \geq 2$) est le nœud de numéro égal à la partie entière du résultat de la division de i par 2. Si donc on dispose d'un tableau de m cases, indicées de 1 à m , pour stocker un tas ayant au plus m nœuds, on peut y stocker l'élément contenu dans le nœud de numéro i dans la case d'indice i . Les fils du nœud de numéro i , s'ils existent, sont stockés dans les cases d'indices $2i$ et $2i + 1$ et son père, s'il existe (c'est-à-dire si le nœud considéré n'est pas la racine), se trouve dans la case d'indice $\lfloor i/2 \rfloor$. C'est ce codage avec un tableau qui sera en général retenu dans le traitement informatique d'un tas. Nous appellerons nœud d'indice i le nœud qui est numéroté par i , et donc qui est rangé dans la i^{e} case du tableau.

On donne ci-dessous la représentation du tas considéré précédemment en exemple par un tableau, les nœuds du tas étant numérotés comme on l'a indiqué plus haut.



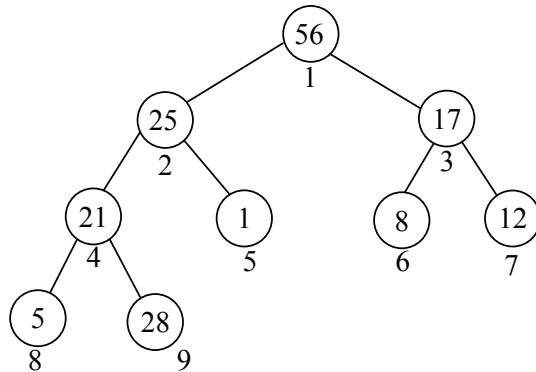
1	2	3	4	5	6	7	8
56	25	17	21	1	8	12	5

Nous allons tout d'abord voir comment structurer une liste donnée de valeurs en tas. À partir d'un tas vide, on insère les éléments les uns après les autres en veillant à conserver la structure de tas. Pour respecter la structure d'arbre binaire parfait on insère le nouvel élément à la première place disponible dans la dernière rangée si celle-ci n'est pas encore pleine ; si la dernière rangée est pleine, on en crée une nouvelle et on insère le nouvel élément à la

première place de la nouvelle rangée. Dans les deux cas, on a inséré le nouvel élément dans la première case non utilisée du tableau. Ensuite, on « fait remonter cet élément » dans l'arbre, en l'échangeant avec son père, tant qu'il est plus grand que son père.

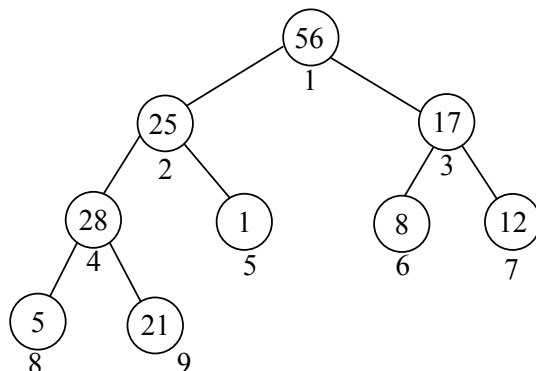
3.8.3 Exemple d'insertion d'un nouvel élément dans un tas

Imaginons que, partant du tas représenté ci-dessus, on veuille maintenant introduire la valeur 28. On place la valeur 28 à droite de la dernière feuille introduite, c'est-à-dire dans la case d'indice 9 du tableau.



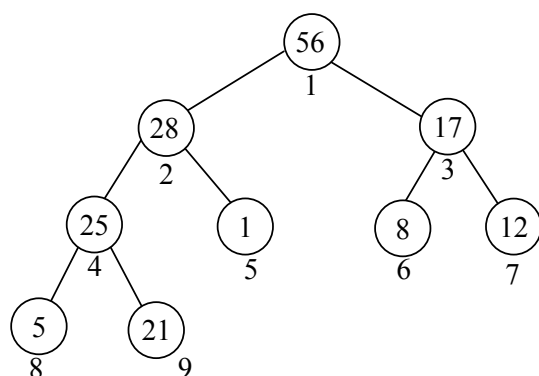
1	2	3	4	5	6	7	8	9
56	25	17	21	1	8	12	5	28

On compare 28, la nouvelle donnée insérée, avec la donnée contenue dans le nœud père, autrement dit on compare la donnée de la case d'indice 9 du tableau avec la donnée de la case d'indice $\lfloor 9/2 \rfloor = 4$. Puisque 28 est plus grand que 21, on échange le 21 et le 28. En observant cet échange, on remarque qu'en toute généralité la seule relation définissant un tas qui peut ne pas être vérifiée est maintenant celle qui existe entre la donnée (28) contenue par le nœud d'indice 4 et la donnée (25) contenue par le nœud père de celui-ci, d'indice $\lfloor 4/2 \rfloor = 2$.



1	2	3	4	5	6	7	8	9
56	25	17	28	1	8	12	5	21

On compare la donnée 28 du nœud d'indice 4 avec la donnée 25 contenue dans son nœud père, d'indice 2. Puisque 28 est plus grand que 25, on échange le 25 et le 28. Comme précédemment, on remarque que la seule relation définissant un tas qui peut ne pas être vérifiée est maintenant celle qui existe entre la donnée (28) contenue par le nœud d'indice 2 et la donnée (56) contenue par le nœud d'indice $\lfloor 2/2 \rfloor = 1$.



1	2	3	4	5	6	7	8	9
56	28	17	25	1	8	12	5	21

On compare la donnée 28 de la case d'indice 2 avec la donnée contenue dans le nœud père d'indice 1. Puisque 28 est plus petit que 56, l'insertion est terminée.

On a bien ainsi restaurée la structure de tas.

3.8.4 Pseudo-code de l'insertion d'une donnée dans un tas

On suppose qu'on dispose d'un tas de $p - 1$ nœuds et qu'on ajoute un nœud d'indice p ; il s'agit de décrire en pseudo-code ce qui a été développé ci-dessus, pour obtenir un tas de p nœuds ; nous nommons *montée* cette procédure. On utilise une variable nommée *clé* qui mémorise la donnée à insérer de façon à éviter des échanges inutiles. Le tableau représentant le tas est noté T . On peut remarquer que cette procédure est très proche de l'insertion séquentielle d'une nouvelle donnée dans un tableau trié ; on se déplace de père en père au lieu de se déplacer d'une case vers celle qui lui est adjacente à gauche.

Procédure *montée*(p)

On utilise un indice entier i et une variable *clé* du même type que les données du tas.

- $i \leftarrow p$;
- $clé \leftarrow T[p]$;
- tant que $i \geq 2$ et que $clé > T[\lfloor i/2 \rfloor]$, faire
 - $T[i] \leftarrow T[\lfloor i/2 \rfloor]$;
 - $i \leftarrow \lfloor i/2 \rfloor$;
- $T[i] \leftarrow clé$;

On peut utiliser cette même procédure si, dans un tas ayant n données, la donnée contenue par le nœud d'indice p ($p \leq n$) a été augmentée. On verra plus loin comment traiter le cas de la diminution d'une donnée du tas.

3.8.5 Complexité d'une insertion dans un tas

Lorsqu'on insère une p^e donnée dans un tas, celle-ci est insérée initialement en dernier dans le tableau, c'est-à-dire dans le niveau de profondeur $h = \lfloor \log_2 p \rfloor$; on fait alors au plus h comparaisons et échanges d'éléments.

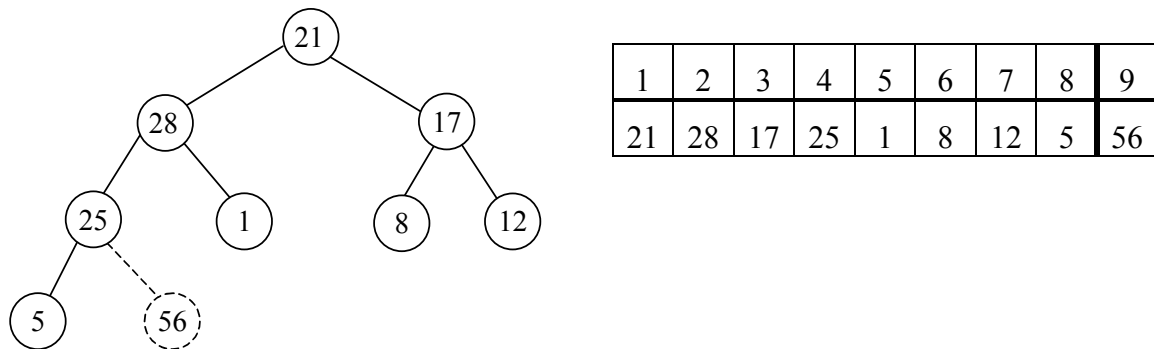
3.8.6 Principe du tri tas

Dans un tas, le plus grand élément est à la racine. Nous allons voir comment tirer parti de cette indication pour trier des données de façon très efficace.

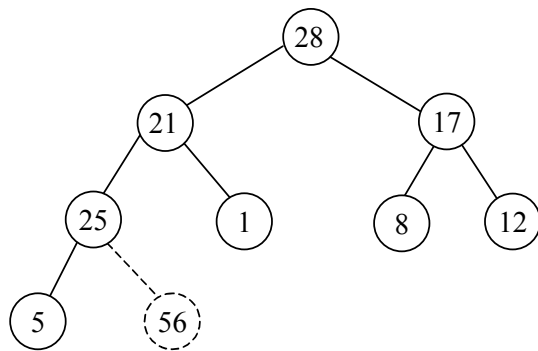
On échange la donnée contenue par la racine avec la donnée contenue dans la dernière feuille du tas, c'est-à-dire avec la donnée contenue dans la dernière case du tableau représentant le tas. La donnée contenue dans la racine avant l'échange étant la plus grande, après l'échange la donnée contenue dans la dernière case du tableau est la plus grande et est donc à sa place dans un tri par ordre croissant ; on ne va plus y toucher. On va alors reconstruire un tas avec les données d'indices compris entre 1 et $n - 1$ (en notant n le nombre initial de données) ; la seule condition définissant un tas qui peut être violée est la supériorité de la donnée de la racine par rapport à ses fils. Pour cela, dans une démarche symétrique de celle de la construction du tas, on « fait descendre » la donnée mise à la racine : tant qu'on n'a pas atteint une feuille du tas et que la donnée qui descend est plus petite que la plus grande des données de ses deux fils, on l'échange avec cette plus grande donnée. Nous illustrons cette suppression de la racine dans le paragraphe ci-dessous.

3.8.7 Exemple de suppression de la racine

On échange la donnée de la racine avec la dernière donnée du tas.

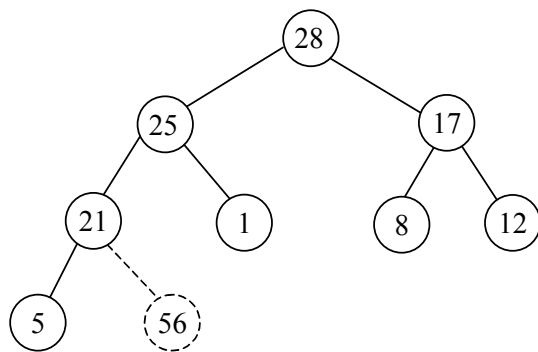


On cherche à reconstituer un tas sur les 8 premières cases du tableau. Pour cela, on cherche la plus grande des deux données contenues dans les fils de la racine, c'est-à-dire les nœuds d'indice $2 \times 1 = 2$ et $2 \times 1 + 1 = 3$; la plus grande de ces données vaut 28 ; on échange donc le contenu de la case 1 avec le contenu de la case 2.



1	2	3	4	5	6	7	8	9
28	21	17	25	1	8	12	5	56

On cherche la plus grande des données entres les données contenues par les nœuds d'indice $2 \times 2 = 4$ et $2 \times 2 + 1 = 5$; la plus grande de ces données vaut 25 et est plus grande que 21 ; on échange donc le contenu de la case 2 avec le contenu de la case 4.



1	2	3	4	5	6	7	8	9
28	25	17	21	1	8	12	5	56

Comme on ne considère qu'un tas sur les 8 premières cases, le nœud d'indice 4 a un seul fils dans le tas, qui contient la donnée 5. Comme 21 est plus grand que 5, la descente de la donnée insérée à la racine est terminée.

3.8.8 Pseudo-code de la descente d'une donnée

Nous allons écrire une procédure un peu plus générale que ce qui a été utilisé plus haut. On suppose qu'on dispose d'un tas de p données et que la donnée qui se trouve dans le nœud d'indice q est diminuée. On souhaite alors réarranger les données pour récupérer la structure de tas sur les p données ; il faut pour cela faire « descendre » la donnée diminuée jusqu'à ce qu'elle soit à la bonne place. Le principe de la procédure correspond à ce qui a été fait plus haut. On utilise une variable nommée *clé* qui mémorise la donnée qui descend, de façon à éviter des échanges inutiles. Le tableau représentant le tas est noté T . Cette procédure est très proche de l'insertion séquentielle d'une nouvelle donnée dans un tableau trié. On se déplace de fils en « plus grand fils » au lieu de se déplacer d'une case vers celle qui lui est adjacente à gauche.

Procédure *descente*(q, p)

On utilise deux indices entiers i et *indice_grand* et deux variables, *clé* et *grand*, du même type que les données du tas, et une variable booléenne *trouvé*.

- $trouvé \leftarrow \text{faux}$;
- $i \leftarrow q$;
- $clé \leftarrow T[q]$;
- tant que (non *trouvé* et $2i \leq p$) faire
 - si $2i = p$, alors $indice_grand \leftarrow p$;
sinon
 - si $T[2i] \geq T[2i + 1]$, alors $indice_grand \leftarrow 2i$;
sinon $indice_grand \leftarrow 2i + 1$;
 - si $clé < T[indice_grand]$, alors
 - $T[i] \leftarrow T[indice_grand]$;
 - $i \leftarrow indice_grand$;
 - sinon $trouvé \leftarrow \text{vrai}$; (* instruction qui sert à sortir de la boucle *)
- $T[i] \leftarrow clé$.

3.8.9 Complexité de la descente de la donnée de la racine

Si on veut utiliser la descente d'une donnée à partir de la racine, la complexité est au pire de l'ordre de la hauteur de l'arbre binaire, puisque, pour chaque niveau de l'arbre, on fait un nombre d'opérations majoré par une constante (au plus deux comparaisons et quelques affectations) ; la descente de la donnée de la racine est donc en $O(\ln n)$ pour un tas de n données.

3.8.10 Pseudo-code du tri tas

On suppose qu'on dispose d'un tableau T contenant n données dans le désordre, dans les cases d'indices compris entre 1 et n , et qu'on veut trier ces données. On peut utiliser la procédure *tri_tas* décrite ci-dessous en pseudo-code.

Procédure *tri_tas*

On utilise un indice entier p .

- Pour p qui varie de 2 à n , faire *montée*(p) ;
- Pour p qui varie de n à 2, faire
 - *échanger*($T, 1, p$) ;
 - *descente*(1, $p - 1$).

3.8.11 Complexité du tri tas

La complexité se calcule directement à partir de celle des procédures *montée* et *descente* ; elle est en $O\left(\sum_{p=1}^n \ln p\right)$, c'est-à-dire en $O(n \ln n)$. On constate donc que ce tri a la complexité optimum d'un tri comparatif.

3.8.12 Exercice

Si vous connaissez ou lorsque vous connaîtrez l'algorithme de Dijkstra, vous pourrez utiliser la structure de tas pour modifier l'algorithme afin d'avoir une complexité en $m \ln n$, en notant n le nombre de sommets du graphe et m son nombre d'arcs.

4. Le hachage

4.1. Principe général

Le hachage peut être utilisé par un analyseur syntaxique pour reconnaître les mots réservés du langage de programmation dans le fichier source d'un programme. Si on ne veut pas faire appel à des structures dynamiques, on doit disposer d'un tableau de taille m , m étant plus grand que le nombre de mots réservés, tableau que nous appellerons *table de hachage*. On définit par ailleurs sur l'ensemble des chaînes de caractères une fonction f à valeurs dans l'ensemble des entiers $\{0, \dots, m - 1\}$ dite *fonction de hachage* ou *de codage*. On indique plus bas des exemples de telles fonctions. On code les mots réservés à l'aide de la fonction f . Si un mot a pour code k , on le range en position k dans la table. Hélas, même si la table est nettement plus grande que le nombre de mots réservés, on peut avoir à faire face à des *collisions*. On dit qu'il y a *collision* si la case du tableau correspondant au code du mot qu'on veut ranger est déjà occupée. Selon la façon dont on traite les collisions, on obtient une méthode de hachage différente ; néanmoins, toutes les méthodes font en sorte qu'on ne déplace pas un mot une fois qu'il occupe une position donnée dans le tableau. Pour simplifier, supposons momentanément qu'on n'ait pas rencontré de collision et qu'on ait donc rangé dans la table tous les mots réservés aux positions attendues. Par la suite, lorsque l'analyseur rencontre un mot du programme, il le code ; si la case correspondante est vide ou contient un mot différent du mot considéré, on sait, en au plus une comparaison, qu'il ne s'agit pas d'un mot réservé ; dans le cas contraire, la réponse est positive.

Nous allons maintenant spécifier trois façons de gérer les collisions et, par suite, la recherche.

Mais illustrons tout d'abord, à l'aide d'un exemple, le fait que les collisions sont « presque » inévitables.

4.2. Exemple pour illustrer le risque de collisions

Dans une classe de 23 élèves, il y a un peu plus d'une chance sur deux pour que deux élèves aient la même date anniversaire ! Ou encore, dit dans le langage de ce chapitre, si on dispose de 365 cases pour coder 23 mots, avec une fonction de codage qui distribue uniformément les mots (la probabilité qu'un mot ait k pour code est égale à $1/365$ pour tout k compris entre 0 et 364), il y a un peu plus d'une chance sur deux pour qu'on rencontre le phénomène de collision.

Preuve

Nous allons calculer la probabilité de l'événement inverse : les 23 codes sont différents. Pour calculer cette probabilité, imaginons qu'on a rangé les mots dans une suite et qu'on les code dans l'ordre de cette suite. Il y a 365^{23} suites différentes, équiprobables (avec l'hypothèse « idéale » de l'uniformité de la répartition des codes). Parmi ces suites, il y en a $365(365 - 1)(365 - 2) \dots (365 - 22)$ qui correspondent à des codages différents. La probabilité pour qu'il n'y ait pas de collision est le rapport de ces deux nombres :

$$R = \prod_{i=0}^{22} \frac{365-i}{365} = \prod_{i=1}^{22} \left(1 - \frac{i}{365}\right).$$

$$\text{D'où : } \ln R = \sum_{i=1}^{22} \ln\left(1 - \frac{i}{365}\right) \approx -\frac{1}{365} \sum_{i=1}^{22} i = -\frac{1}{365} \frac{22 \times 23}{2} \approx -0,6932.$$

Or on a $\exp(-0,6932) \approx 0,4999$, ce qui donne une approximation de la probabilité qu'il n'y ait pas de collision. Celle qu'il y ait des collisions vaut donc un peu plus de 0,5.

4.3. Exemples de fonctions de hachage

Les fonctions de hachage que nous donnons en exemple sont définies sur l'ensemble des chaînes de caractères. On peut aussi définir des fonctions de hachage sur toute sorte d'ensembles, par exemple sur l'ensemble des entiers.

Fonction h_1 : cette fonction ne convient que pour les chaînes écrites en minuscules avec les 26 caractères de l'alphabet. On code chaque lettre de l'alphabet par son rang dans l'alphabet, 'a' ayant ainsi le code 1. On associe à chaque mot tout d'abord la somme s des codes de ses lettres, puis le reste modulo m de s .

Exemple : si $m = 20$, pour la chaîne "lou", on obtient :

$$h_1(\text{"lou"}) = (12 + 15 + 21) \text{ modulo } 20 = 8.$$

Les valeurs de cette fonction seront mal réparties sur l'ensemble $\{0, \dots, m - 1\}$ si m est grand et que les chaînes sont courtes.

Fonction h_2 : on code chaque lettre de la chaîne par son code ASCII ; on multiplie ce code ASCII par la position de la lettre dans la chaîne considérée ; on additionne les valeurs obtenues pour chacune des lettres puis on prend le reste modulo m .

Exemple : si $m = 100$, pour la chaîne "lou", on obtient :

$$h_2(\text{"lou"}) = (108 \times 1 + 111 \times 2 + 117 \times 3) \text{ modulo } 100 = 81.$$

Ce code a l'avantage de tenir compte de la position des lettres dans le mot.

Fonction h_3 : on peut faire beaucoup plus sophistiqué ; la fonction h_3 utilise différents principes qui ne sont souvent que partiellement utilisés par d'autres fonctions de hachage. On choisit un entier t qui ne doit pas être trop grand de façon à éviter ci-dessous la manipulation de trop grands entiers (voir ci-dessous l'utilisation de t) ; on choisit un réel θ vérifiant $0 < \theta < 1$; un bon choix de θ peut être $\theta = (\sqrt{5}-1)/2 \approx 0,6180339887$ ou bien $\theta = 1 - (\sqrt{5}-1)/2 \approx 0,3819660113$. Si x est un nombre réel, on note ci-dessous $x \bmod 1$ la partie décimale de x , c'est-à-dire la valeur obtenue en remplaçant par 0 la partie qui se trouve « avant la virgule ».

Pour obtenir la valeur de $h_3(s)$, où s est une chaîne de caractères, on effectue les opérations suivantes :

1. on code chaque lettre de la chaîne s par son code ASCII ;
2. on écrit en binaires les valeurs obtenues (le chiffre de gauche est un 1) ;
3. on concatène les chaînes binaires de ces écritures ;
4. on complète éventuellement la chaîne par des 0 à droite pour avoir une chaîne dont la longueur soit multiple de t ;
5. on découpe la chaîne concaténée en tranches de longueur t ;
6. on effectue un « ou exclusif » bit à bit sur l'ensemble de ces morceaux de chaînes ;
7. on appelle v la valeur, écrite en décimale, du résultat du « ou exclusif » ;
8. on pose $h_3(s) = \lfloor ((v \times \theta) \bmod 1) \times m \rfloor$.

On remarque qu'à l'issue du point 4, deux chaînes différentes donneront des résultats différents.

Exemple : pour $t = 8$, $\theta = (\sqrt{5} - 1)/2$ et $m = 100$, pour la chaîne "lou", on obtient successivement:

1. 'l' $\rightarrow$ 108 ; 'o' $\rightarrow$ 111 ; 'u' $\rightarrow$ 117 ;
2. 108 $\rightarrow$ 1101100 ; 111 $\rightarrow$ 1101111 ; 117 $\rightarrow$ 1110101 ;
3. 110110011011111110101
4. 110110011011111110101000
5. 11011001, 10111111, 10101000
6. 11001110
7. $v = 206$; $v \times \theta \approx 127,315$
8. $h_3(\text{"lou"}) = 31$.

4.4. Le hachage linéaire

Si, lors de la construction de la table, on veut ranger un nouveau mot et que la case correspondant au code de ce mot est déjà occupée, on se déplace dans la table, à partir de cet indice « cycliquement vers la droite » jusqu'à ce qu'on trouve une place vide. On y place alors l'élément. Dire qu'on se déplace « cycliquement vers la droite » signifie qu'on se déplace vers la droite mais que lorsqu'on arrive à la position $m - 1$ et que celle-ci est occupée, on repart de la position 0.

On dit qu'on a une *collision primaire* lorsque deux mots ont le même code, et donc se voient initialement attribuer la même place dans le tableau. Remarquons qu'ici on peut rencontrer des *collisions secondaires*, c'est-à-dire le fait que deux mots se disputent une même case, sans avoir cependant le même code, par exemple parce que le premier à être entré dans la table avait dû être déplacé vers la droite, la place lui revenant étant déjà occupée.

Exemple : supposons que les chaînes "lou", "david", "sophie", "marie", "jean", "gaspard" doivent être entrées dans une table de hachage ayant $m = 7$ cases avec :

$$h(\text{"lou"}) = 5, h(\text{"david"}) = 4, h(\text{"sophie"}) = 2, h(\text{"marie"}) = 4, h(\text{"jean"}) = 0, h(\text{"gaspard"}) = 6.$$

Les chaînes "lou", "david" et "sophie" vont dans les cases d'indice correspondant à leurs codes ; on a une collision primaire pour la chaîne "marie" qui devrait aller dans la case d'indice 4 qui est occupée, on essaie alors pour "marie" la case d'indice 5, mais cette case est occupée par "lou" ; on essaie enfin la case d'indice 6 qui est libre, on y met "marie" ; on place

"jean" sans problème ; lorsqu'on veut placer "gaspard", on a une collision secondaire sur la case 6 avec "marie" ; on essaie alors la case d'indice 0, qui est occupée par "jean" et on place "gaspard" dans la case d'indice 1. On a donc obtenu :

0	1	2	3	4	5	6
"jean"	"gaspard"	"sophie"		"david"	"lou"	"marie"

Comment s'effectue alors la recherche d'un élément dans la table ? On code l'élément, on se place dans la table à la position d'indice égal au code de cet élément, on compare l'élément avec celui qui est contenu à cette place, puis on se déplace cycliquement vers la droite dans la table, jusqu'à ce qu'on trouve l'élément cherché (ceci pouvant avoir lieu dès la première comparaison) ou que l'on trouve une place non occupée ou, si le tableau est complètement plein, jusqu'à ce qu'on soit revenu au point de départ (ces deux derniers cas sont les cas d'échec de la recherche).

4.5. Le hachage avec chaînage interne

Comme dans le hachage linéaire, on place tous les éléments dans un tableau. Ici, l'idée est, d'une part, de prévoir dans le tableau, en plus des m cases, une zone de débordement dans laquelle on rangera les éléments dont la place légitime était occupée lorsqu'on a voulu les ranger et, d'autre part, de « chaîner » entre eux les éléments qui entrent en collision primaire. On note p la taille de la zone de débordement ; cette zone se situe entre les indices m et $m + p - 1$ du tableau. Lors de la construction de la table, on prévoit pour chaque indice deux champs : un champ pour l'élément à ranger et un champ pour indiquer éventuellement un indice du tableau ; cette seconde information servira à faire un chaînage interne des éléments dépendant d'une même case ; une valeur de -1 dans le second champ indiquera qu'il n'y a pas d'« élément suivant ». Supposons que l'élément en cours de rangement ait pour code k ;

- si la case d'indice k est libre, on met l'élément à la place k et la valeur -1 dans le second champ ;
- si la case d'indice k est occupée, on range alors l'élément à la première case libre à **partir de la droite** du tableau, case qui est dans la zone de débordement si celle-ci n'est pas déjà pleine et la valeur -1 dans le second champ de cette même case. On note i l'indice de la case où on vient de ranger le nouvel élément. Si le second champ de la case d'indice k vaut -1 , on remplace cette valeur par la valeur i ; si ce second champ ne valait pas -1 , c'est qu'il y a déjà eu une collision sur la case k ; on parcourt alors la chaîne interne issue de la case k en suivant les indices successifs donnés par les seconds champs jusqu'à rencontrer un second champ de valeur -1 , valeur qu'on remplace alors par la valeur i : tous les éléments de code k sont chaînés dans le tableau à partir de la case d'indice k .

Si la zone de débordement est pleine, on utilise la table initiale comme zone de débordement en occupant la première place libre à partir de la droite du tableau. Ceci ralentit la fabrication initiale de la table, car cela introduit des collisions secondaires.

Exemple : on choisit $m = 7$ et $p = 2$; supposons que l'on ait à placer des éléments $e_1, e_2, e_3, e_4, e_5, e_6$ et e_7 avec $h(e_1) = 4, h(e_2) = 6, h(e_3) = 2, h(e_4) = 4, h(e_5) = 2, h(e_6) = 2, h(e_7) = 5$. On place d'abord e_1, e_2 et e_3 ; on obtient :

0	1	2	3	4	5	6	7	8
		e_3		e_1		e_2		
		-1		-1		-1		

La place de e_4 n'est pas libre ; on a une collision primaire et le tableau devient après insertion de e_4 :

0	1	2	3	4	5	6	7	8
		e_3		e_1		e_2		e_4
		-1		8		-1		-1

La place de e_5 n'est pas libre ; on a une collision primaire et le tableau devient après insertion de e_5 :

0	1	2	3	4	5	6	7	8
		e_3		e_1		e_2	e_5	e_4
		7		8		-1	-1	-1

La place de e_6 n'est pas libre ; on a une collision primaire, la zone de débordement est pleine et la première place libre en partant de la droite a pour indice 5 ; le tableau devient après insertion de e_6 :

0	1	2	3	4	5	6	7	8
		e_3		e_1	e_6	e_2	e_5	e_4
		7		8	-1	-1	5	-1

Les éléments de code 2 sont chaînés et occupent successivement les places d'indices 2, 7 et 5.

La place de e_7 , d'indice 5, n'est pas libre et elle est occupée par un élément dont le code ne vaut pas 5 ; on a une collision secondaire ; le tableau devient après insertion de e_7 :

0	1	2	3	4	5	6	7	8
		e_3	e_7	e_1	e_6	e_2	e_5	e_4
		7	-1	8	3	-1	5	-1

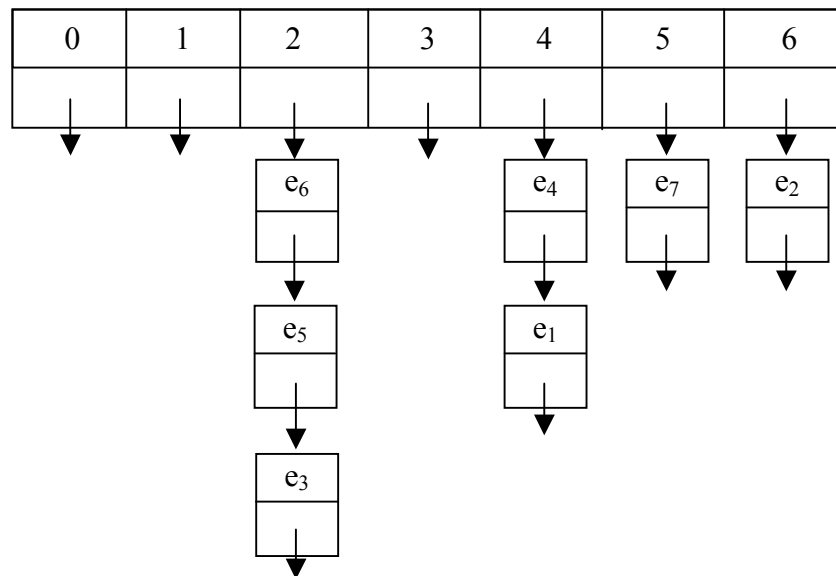
La recherche d'un élément x se déroule comme suit dans le cas du hachage avec chaînage interne. On code x ; appelons k le code de x . Si la case d'indice k ne contient pas d'élément, on conclut à l'échec ; sinon on compare successivement x à tous les éléments qui sont chaînés à partir de la case d'indice k jusqu'à avoir trouvé l'élément (situation de succès de la recherche) ou bien avoir atteint la fin de cette chaîne (situation de l'échec de la recherche).

4.6. Le hachage avec chaînage externe

Lorsqu'on peut utiliser de l'allocation dynamique de mémoire, on pourra préférer le chaînage externe pour gérer les collisions.

La méthode est simple : chaque case du tableau est le début d'une liste chaînée qui contient les éléments de la table dont le code est égal à l'indice de cette case ; les cases du tableau ne contiennent donc ici que les adresses en mémoire des débuts des listes chaînées ou bien l'adresse nulle si aucun élément n'a l'indice de la case comme code. Lors de l'utilisation de la table, pour voir si un « mot » est ou non dans la table, on code ce mot, puis on parcourt la liste des éléments chaînés dans la case d'indice égal au code trouvé, jusqu'à ce qu'on trouve l'élément (situation de succès de la recherche) ou que l'on arrive à la fin de la liste sans l'avoir trouvé (situation de l'échec de la recherche).

Rappelons l'exemple du paragraphe précédent. Les éléments sont : $e_1, e_2, e_3, e_4, e_5, e_6$ et e_7 avec $h(e_1) = 4$, $h(e_2) = 6$, $h(e_3) = 2$, $h(e_4) = 4$, $h(e_5) = 2$, $h(e_6) = 2$, $h(e_7) = 5$. On utilise un tableau de longueur $m = 7$. On obtient le tableau suivant :



4.7. Nombre de comparaisons

Nous ne calculons pas ici le nombre de comparaisons effectuées en moyenne pour une insertion ou une recherche dans une table de hachage et nous nous contentons de quelques indications.

Signalons que la technique du hachage linéaire est moins performante que les deux autres techniques et que, dans ce cas, il faut prendre un tableau largement plus grand que le nombre de données à ranger ; en effet, on peut remarquer que, lorsqu'il y a des plages de cases consécutives occupées, alors, plus la plage est large, plus la probabilité qu'une prochaine donnée à insérer ait son code dans cette plage est grande ; les plages les plus grandes ont tendance à s'agrandir davantage.

Les deux techniques avec chaînages ont des performances équivalentes si la zone de débordement du hachage avec chaînage externe est prévue suffisamment grande. Le hachage externe est marginalement meilleur quand aucune zone de débordement n'est prévue.

On dit qu'une recherche est positive si la donnée cherchée figure dans la table et qu'elle est négative dans le cas contraire. Le nombre moyen d'accès au tableau pour rechercher une donnée est toujours un peu plus grand pour une recherche négative que pour une recherche positive. Quand on compte le nombre moyen d'accès au tableau, on inclut l'accès éventuel

qui indique que la chaîne est terminée. Les évaluations ci-dessous restent correctes même si la zone de débordement du hachage avec chaînage interne a une taille nulle.

Si le nombre de cases du tableau est deux fois plus grand que le nombre de données à ranger, le nombre moyen d'accès au tableau pour rechercher une donnée est compris entre 1,2 et 1,4 pour une recherche positive et entre 1,5 et 1,7 pour une recherche négative dans le cas des hachages avec chaînage. Pour le hachage linéaire, il est de l'ordre de 1,5 pour une recherche positive et de 3 pour une recherche négative.

Si le nombre de cases du tableau est égal à 1,5 fois le nombre de données à ranger, le nombre moyen d'accès au tableau pour rechercher une donnée est compris entre 1,3 et 1,5 pour une recherche positive et entre 1,6 et 1,9 pour une recherche négative dans le cas des hachages avec chaînage. Pour le hachage linéaire, il est de l'ordre de 2,5 pour une recherche positive et de 6 pour une recherche négative.

Si le nombre de cases du tableau est égal à 1,1 fois le nombre de données à ranger, le nombre moyen d'accès au tableau pour rechercher une donnée est compris entre 1,4 et 1,7 pour une recherche positive et entre 1,9 et 2,2 pour une recherche négative dans le cas des hachages avec chaînage. Le hachage linéaire devient impraticable.

5. L'algorithme de Huffman

5.1. Présentation du problème

L'algorithme de Huffman est utilisé en « codage de source ». Il s'agit de coder les caractères d'un texte à l'aide de suites de 0 et de 1. On appellera *mot de code* le codage d'un des caractères ; un mot de code est donc une chaîne binaire écrite avec des 0 et des 1.

Une première possibilité est d'utiliser un codage où tous les mots de code ont la même longueur ; on dira alors qu'il s'agit d'un *codage de longueur fixe*. Si le texte est écrit avec n caractères différents, il faudra avoir des mots de code de longueur $p = \lceil \log_2 n \rceil$. Prenons un exemple ; supposons que le texte soit écrit uniquement avec les caractères a, b, c, d, e, f, g et h et contiennent 5852 caractères. Étant donné qu'il n'y a que 8 caractères à coder, on peut les coder avec des chaînes binaires distinctes de longueur 3 ; le texte codé sera alors pour longueur (comptée en nombre de 0 et de 1) égale à $3 \times 5852 = 17556$. Le texte codé est facile à décoder puisqu'il suffit de découper le texte codé en mots de longueur p .

On désire, pour diverses raisons (place de stockage, durée de transmission, risque d'erreur de transmission...), avoir un texte codé le plus court possible. Pour cela, on abandonne l'idée du codage de longueur fixe et on va faire en sorte que les caractères les plus fréquents aient des codages plus courts que les caractères plus rares. Cela nécessite d'avoir :

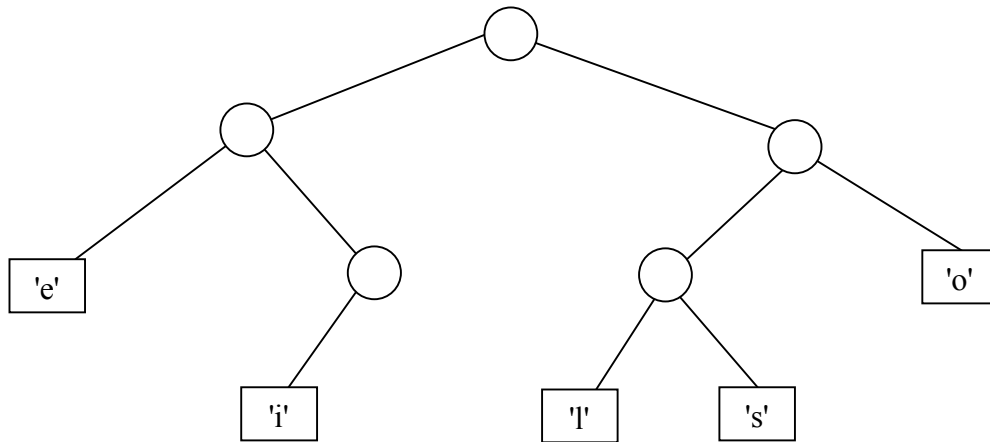
- soit une estimation de la fréquence des caractères dans le type de texte qu'on doit coder ; on pourrait par exemple faire des statistiques sur la fréquences des différents caractères dans l'ensemble des textes écrits en français et utiliser ces statistiques pour le codage d'un texte écrit en français ;
- soit calculer précisément les nombres d'occurrences de chacun des caractères dans le texte que l'on souhaite coder.

Il s'agit donc maintenant, muni de ces informations, de définir un codage de longueur variable. Pour cela, il faut se préoccuper du décodage ; il faudra savoir où s'arrête chacun des mots de code. On peut utiliser un séparateur, mais il est possible de ne pas y recourir dans le cas où le codage respecte la *règle du préfixe* : un ensemble de mots de code vérifie la règle du préfixe si un mot de code n'est jamais préfixe d'un autre mot de code. Par exemple, si on considère les mots de code 0100110 et 010, le second est préfixe du premier ; ces deux mots ne pourront donc pas faire partie simultanément d'un codage respectant la règle du préfixe. Un codage qui suit la règle du préfixe est appelé *codage préfixe*. Montrons que si un codage est préfixe, alors le décodage est possible.

On place les mots de code dans un arbre binaire construit selon l'exemple suivant. On suppose qu'on a cinq caractères 'e', 'i', 'l', 'o' et 's' et que le code est donné par :

$\text{code}('e') = 00$, $\text{code}('i') = 010$, $\text{code}('l') = 100$, $\text{code}('o') = 11$, $\text{code}('s') = 101$.

On vérifie facilement qu'il s'agit d'un codage préfixe ; on construit alors l'arbre binaire suivant :



Les caractères sont les feuilles de l'arbre binaire ; un caractère peut être retrouvé dans l'arbre binaire en partant de la racine et en suivant « le chemin indiqué par le code de ce caractère » : un 0 indique de descendre à gauche et un 1 de descendre à droite. Voyons comment on peut décoder le message : 1011110000010100 en utilisant cet arbre. On part de la racine de l'arbre et on lit le premier chiffre du message qui est un 1 : on part à droite dans l'arbre ; le chiffre suivant est un 0, on continue la descente en partant à gauche, le chiffre suivant est un 1, on part à droite et on atteint le caractère 's', qui est ainsi le premier caractère du texte décodé ; on repart de la racine et on lit le chiffre suivant dans le message, c'est-à-dire le quatrième, il s'agit d'un 1, on part à droite, puis encore un 1, on part à droite et on lit le caractère 'o' ; le message commence par "so". Nous laissons le lecteur terminer le décodage et découvrir le message.

Pour un codage non préfixe, on peut encore tracer un arbre analogue, mais certains caractères se trouvent alors dans des nœuds internes ; si, pendant le décodage, on rencontre un tel caractère, on ne peut pas savoir si on a obtenu ainsi un caractère du texte ou bien s'il faut passer son chemin pour aller voir bas.

On peut montrer que, si on se restreint aux codages d'un texte par le codage des caractères (et non pas par exemple de sous-chaînes), le codage de taille minimum est obtenu par un codage préfixe. Le codage de Huffman que nous allons définir plus bas est un codage préfixe qui permet d'obtenir le texte codé de taille minimum.

Si la règle du préfixe est vérifiée, chaque caractère est associé à une feuille de l'arbre, et la longueur (en nombre de bits) de la chaîne codant le caractère est la distance de la racine à cette feuille. Notre but est d'associer à la suite des caractères à coder un arbre binaire tel que, toute feuille étant associée à un caractère c_i de fréquence f_i et étant à une distance l_i de la racine, la somme des produits $l_i f_i$ prise sur toutes les feuilles soit minimum. Cet arbre sera dit *arbre optimum*.

5.2. Quelques remarques préliminaires

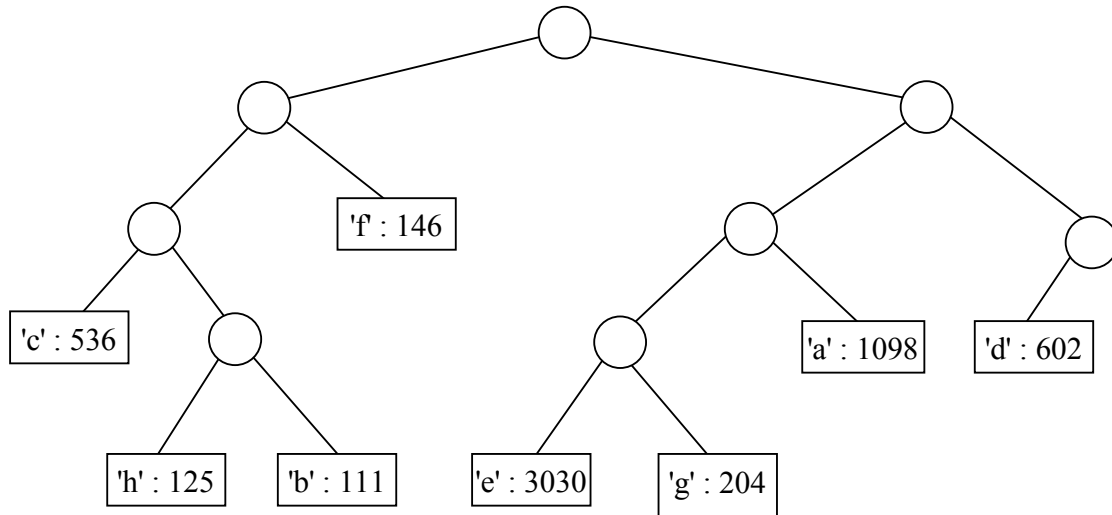
On peut d'abord remarquer que l'opération définie dans le paragraphe précédent qui associe à un codage préfixe un arbre binaire permet de définir une bijection entre les codages

préfixes d'un ensemble de caractères et les arbres binaires dont les feuilles contiennent exactement ces caractères.

Considérons un texte composé des caractères distincts $c_1, c_2, \dots, c_n$, le caractère c_i apparaissant avec un nombre d'occurrences noté $occ(c_i)$. On pourrait considérer les fréquences en divisant les nombres d'occurrences par la longueur du texte à coder, cela reviendrait au même. On considère un codage préfixe des caractères $c_1, c_2, \dots, c_n$; on complète l'arbre binaire correspondant à ce codage en ajoutant à chaque feuille, en plus du caractère x correspondant à cette feuille, le nombre d'occurrences de x ; on note A l'arbre obtenu ; on note $p(x)$ la longueur du code du caractère x ou, ce qui revient au même la profondeur du caractère x dans l'arbre A . Reprenons notre premier exemple du texte de longueur 5852 écrit uniquement avec les caractères 'a', 'b', 'c', 'd', 'e', 'f', 'g' et 'h'. On suppose que l'on a :

$$\begin{aligned} occ('a') &= 1098, occ('b') = 111, occ('c') = 536, occ('d') = 602, \\ occ('e') &= 3030, occ('f') = 146, occ('g') = 204, occ('h') = 125. \end{aligned}$$

Ces données sont des données issues d'un texte réel sauf que ce texte contient aussi d'autres caractères. On peut alors considérer le codage défini par l'arbre A ci-dessous :



Ce codage correspond à :

$$\begin{aligned} \text{code}('a') &= 101, \text{code}('b') = 0011, \text{code}('c') = 000, \text{code}('d') = 110, \\ \text{code}('e') &= 1000, \text{code}('f') = 01, \text{code}('g') = 1001, \text{code}('h') = 0010. \end{aligned}$$

La longueur du texte codé sera alors égale à :

$$1098 \times 3 + 111 \times 4 + 536 \times 3 + 602 \times 3 + 3030 \times 4 + 146 \times 2 + 125 \times 4 + 204 \times 4 = 20080.$$

Si f est une feuille de A , on note $occ(f)$ le nombre d'occurrences contenu par cette feuille. Posons :

$$L(A) = \sum_{f, \text{feuille de } A} occ(f) \times p(f).$$

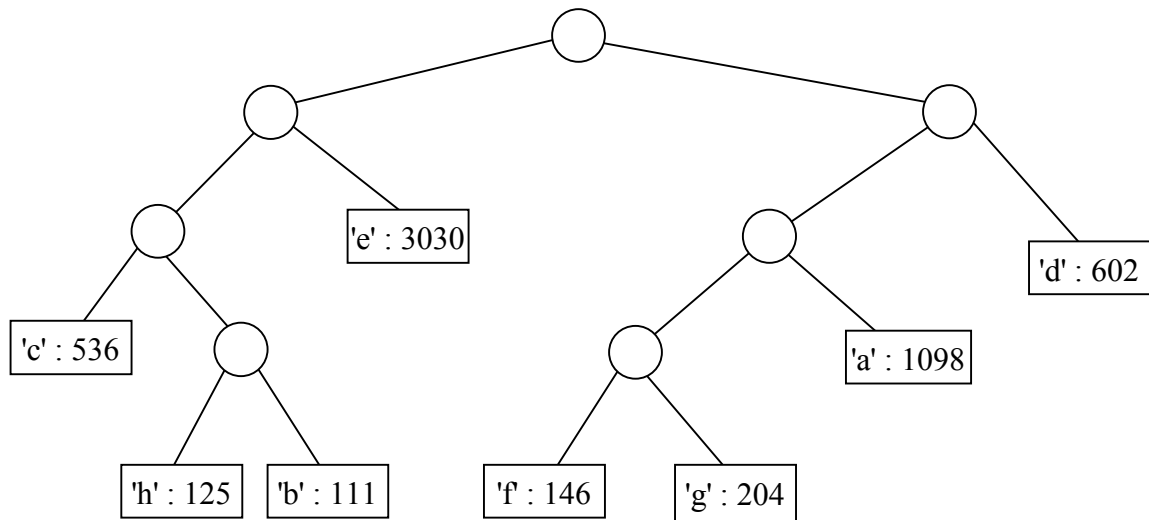
Donc $L(A)$ donne la longueur du texte codé dans le codage correspondant à A .

Le problème devient : déterminer un arbre A qui minimise $L(A)$. Un tel arbre sera dit optimum.

Examinons l'arbre de notre exemple. Peut-il être optimum ? On peut faire les deux remarques suivantes :

- on gagnerait en remontant la feuille contenant 'd' et en la mettant à la place de son père ; on gagnerait exactement 602 ;
- on gagnerait en échangeant la feuille contenant 'f' avec la feuille contenant 'a', on gagnerait exactement $(3030 - 146) \times 2 = 5768$.

En appliquant ces deux transformations, on obtient l'arbre ci-dessous pour lequel L vaut 13710 :



On peut généraliser ces remarques.

Lemme 1 : dans un arbre optimum, tout nœud interne a deux fils.

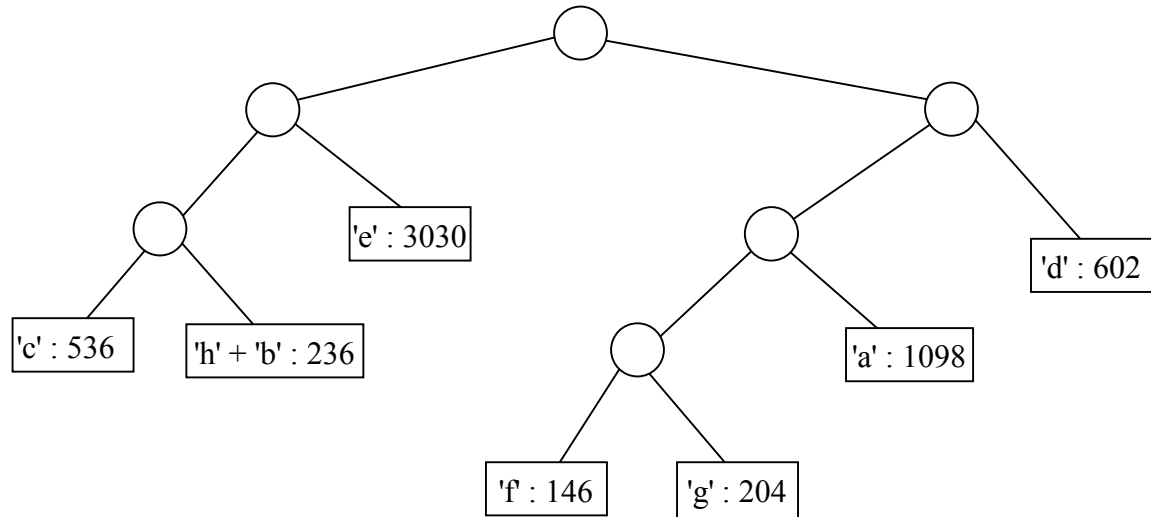
Lemme 2 : dans un arbre optimum, les deux plus petites occurrences se trouvent à la profondeur maximum de l'arbre.

Les preuves de ces deux lemmes sont immédiates. Le second lemme utilise le premier qui montre que, dans un arbre optimum, il y a au moins deux nœuds à la profondeur maximum.

Lemme 3 : il existe un arbre optimum dans lequel les deux plus petites occurrences se trouvent dans des feuilles « frères » qui se trouvent à la profondeur maximum.

En effet, considérons un arbre optimum qui ne vérifie pas la propriété du lemme 3 ; le lemme 2 indique que les deux plus petites occurrences se trouvent à la profondeur maximum de l'arbre et le lemme 1 que toute feuille a une feuille « frère ». Un simple échange permet d'obtenir un arbre qui vérifie la propriété du lemme 3 avec une valeur inchangée de la fonction L .

On considère la transformation suivante de l'arbre A . On considère deux feuilles « frères » f_1 et f_2 . On note x le caractère contenu par f_1 et y le caractère contenu par f_2 . On supprime ces deux feuilles et on transforme leur père en une feuille contenant la chaîne symbolique $x + y$ assorti d'un nombre d'occurrences égal à $occ(x) + occ(y)$; on note A' l'arbre ainsi obtenu. En appliquant cette transformation aux feuilles contenant les caractères 'b' et 'h' du dernier arbre tracé, on obtient l'arbre A' ci-dessous :



On remarque facilement sur l'exemple : $L(A') = L(A) - 236$. Ce résultat se généralise immédiatement à : $L(A') = L(A) - (occ(x) + occ(y))$.

Lemme 4 : On considère un arbre A où les deux plus petites occurrences sont « frères » à la profondeur maximum ; on note x et y les caractères contenus par ces deux feuilles. On effectue l'opération qui transforme A en A' en utilisant ces deux feuilles. L'arbre A est optimum si et seulement si l'arbre A' est optimum.

Preuve du lemme 4

Supposons que A ne soit pas optimum. Considérons un autre arbre binaire B sur le même jeu de données que A vérifiant $L(B) < L(A)$ et où les feuilles contenant x et y sont « frères » à la profondeur maximum (ce qui est possible d'après le lemme 3). On construit l'arbre B' à partir de B . On a :

$$L(A') = L(A) - (occ(x) + occ(y))$$

$$L(B') = L(B) - (occ(x) + occ(y))$$

et donc $L(B') < L(A')$; l'arbre A' n'est donc pas optimum.

Supposons que A' ne soit pas optimum. Considérons un autre arbre binaire B' sur le même jeu de données que A' vérifiant $L(B') < L(A')$. On remplace la feuille contenant la chaîne $x + y$ en effectuant la transformant inverse de celle considérée plus haut : cette feuille devient un nœud interne ayant pour fils deux feuilles contenant x et y avec leurs occurrences respectives. On obtient un arbre B vérifiant : $L(B) = L(B') + (occ(x) + occ(y))$. De la relation $L(A) = L(A') + (occ(x) + occ(y))$, on déduit que $L(B) < L(A)$; l'arbre A n'est donc pas optimum.

Ce résultat est à la base de l'algorithme décrit ci-dessous.

5.3. Description de l'algorithme de Huffman

On construit un arbre réduit à un nœud pour chacun des caractères, ce nœud contenant le caractère et son nombre d'occurrences. On range dans un tableau A (indiqué à partir de 1) les nœuds par ordre décroissant de nombres d'occurrences. On considère les nœuds situés en $A[n - 1]$ et $A[n]$; ils contiennent les plus petits nombres d'occurrences ; on note x et y les caractères contenus par ces nœuds ; on construit un nouveau nœud noté N qui contient la chaîne $x + y$ et la somme des nombres d'occurrences de ces deux caractères ; on donne à N comme fils gauche $A[n - 1]$ et comme fils droit $A[n]$ puis on remplace $A[n - 1]$ par N . On

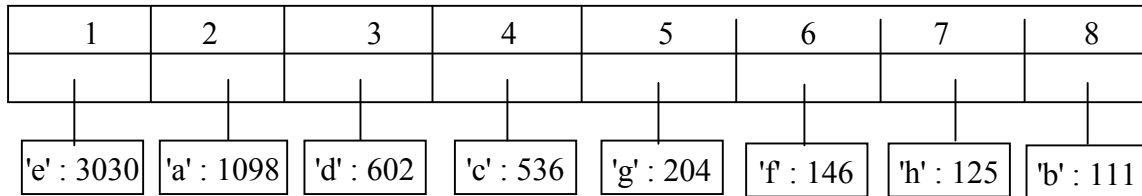
considère qu'il ne reste plus que $n - 1$ arbres dans le tableau A rangés de l'indice 1 à l'indice $n - 1$; on remanie le tableau A pour qu'il soit à nouveau trié par ordre décroissant des nombres d'occurrences, ce qui revient à insérer le nœud N dans la sous-liste triée qui le précède. On procède ainsi jusqu'à ce qu'il n'y ait plus qu'un arbre dans la liste. On obtient alors un codage optimal, comme l'énonce le théorème suivant.

Théorème : L'algorithme de Huffman permet de construire un codage préfixe optimal.

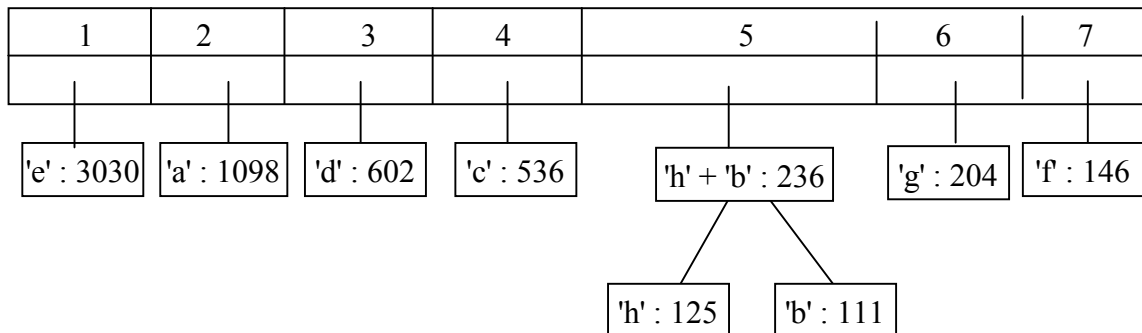
La preuve de ce théorème (et donc de l'algorithme de Huffman) est laissée en exercice et découle des lemmes précédents.

5.4. Exemple d'application

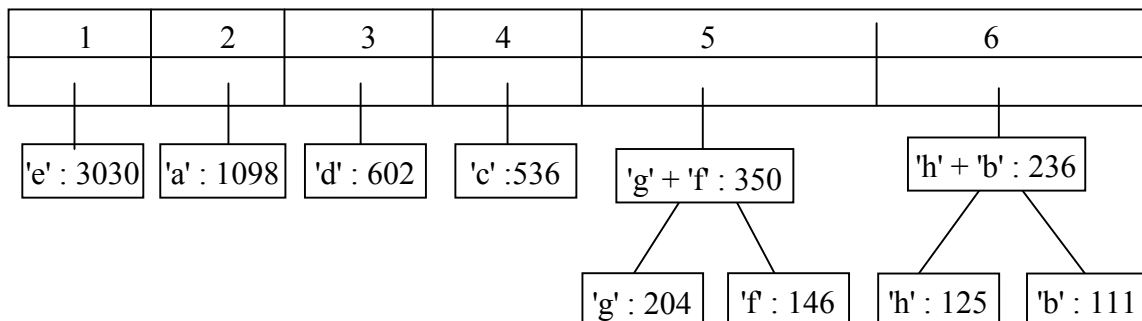
On reprend l'exemple précédent à ses débuts en classant les nombres d'occurrences par ordre décroissant ; on obtient :



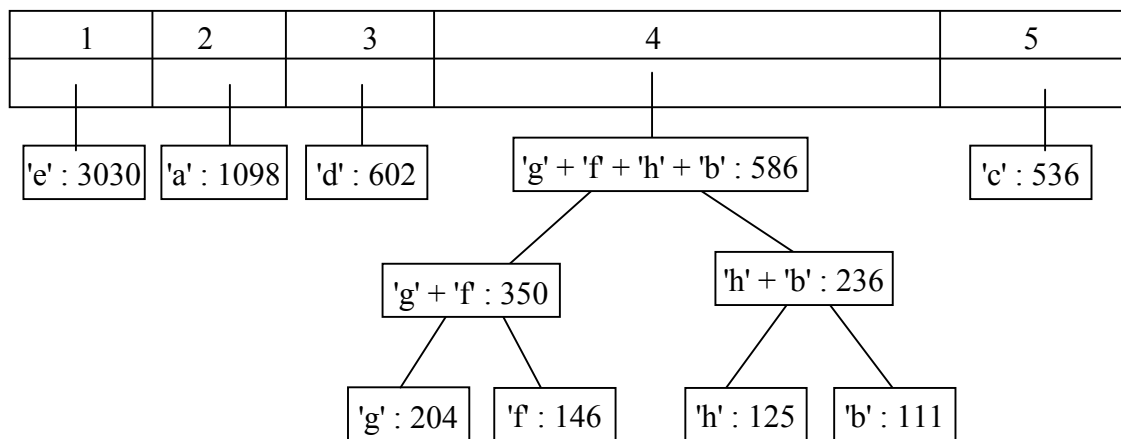
On effectue la première étape de l'algorithme :



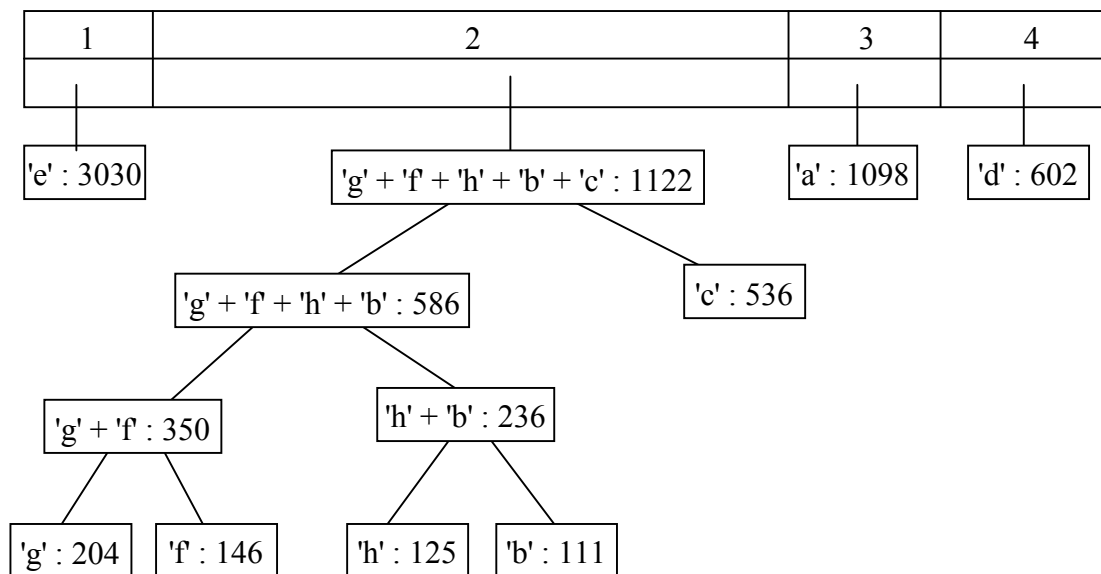
puis l'étape suivante :



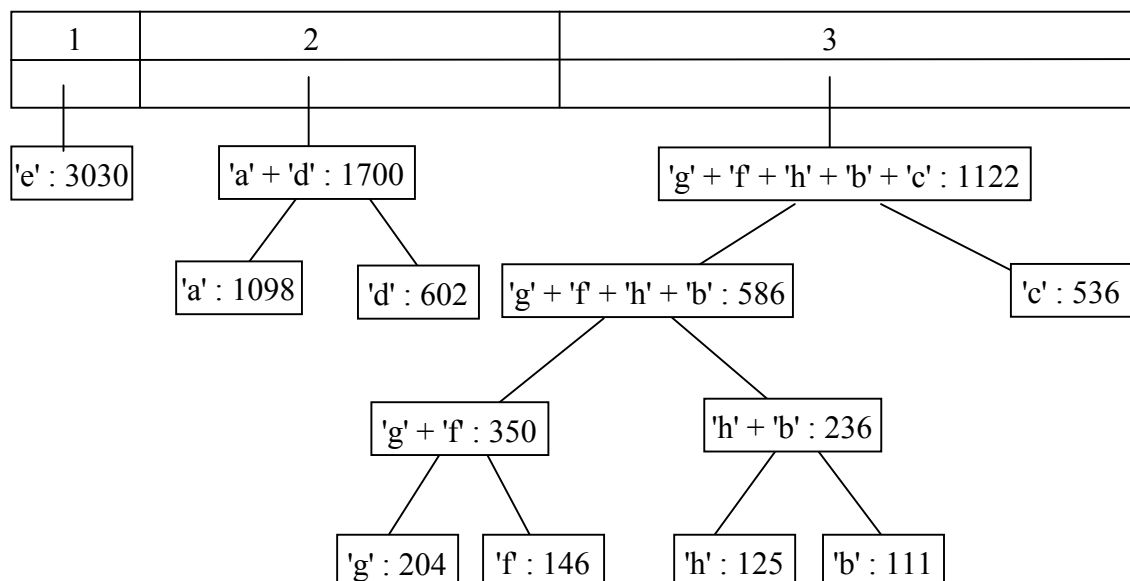
puis :



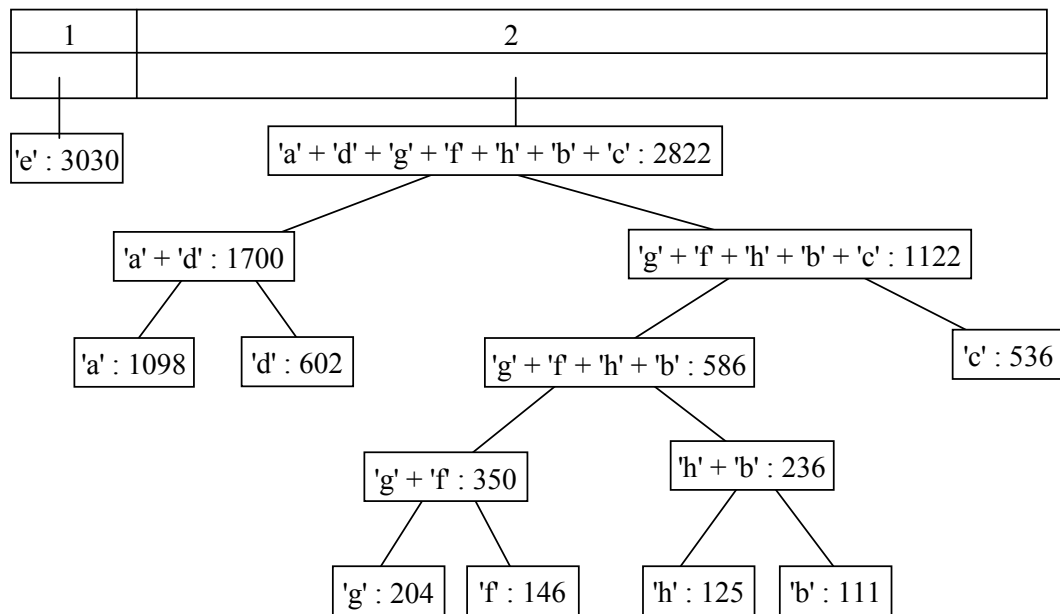
puis :



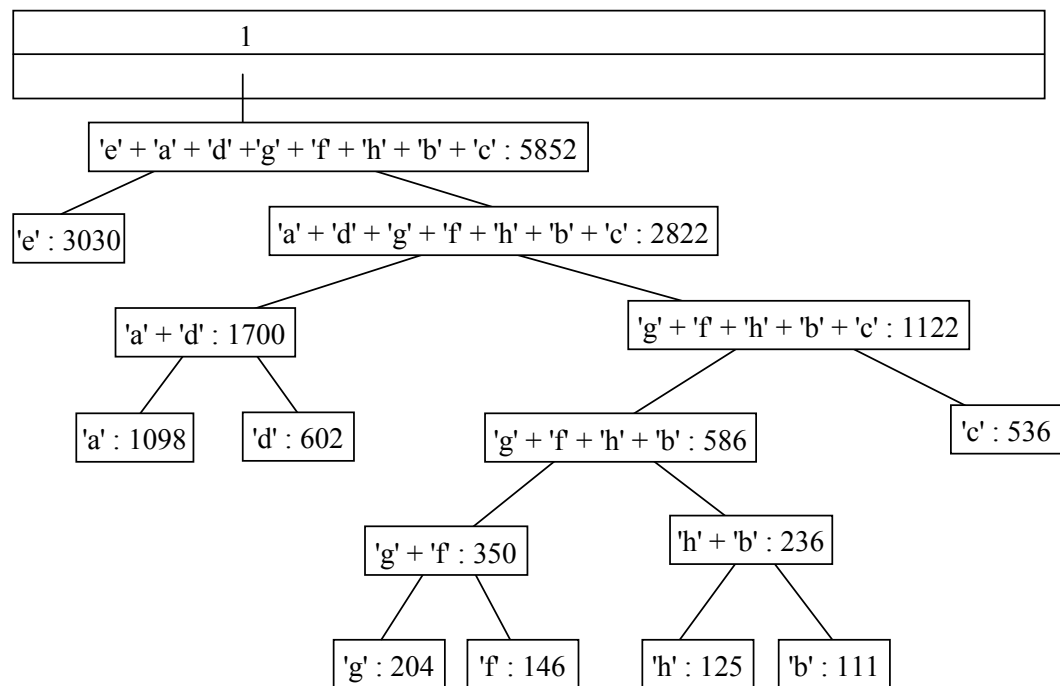
puis :



puis :



et enfin :



Cet arbre donne un code préfixe optimum :

code(a) = 100, code(b) = 11011, code(c) = 111, code(d) = 101,
code(e) = 0, code(f) = 11001, code(g) = 11000, code(h) = 11010.

Avec ce code, le texte codé a pour longueur :

$$(204 + 146 + 125 + 111) \times 5 + (1098 + 602 + 536) \times 3 + 3030 \times 1 = 12668.$$

Tout autre codage des caractères du texte donnera une longueur au moins aussi grande.

L'utilisation d'un tas permet d'obtenir une complexité en $O(n \ln n)$, s'il y a n caractères à coder.

Index

- adresse, 5
 - nulle, 8
- algorithme, 1
 - de Huffman, 43
- ancêtre, 11
- arbre, 10
 - binaire, 13
 - binaire de recherche, 26
 - parfait, 29
- champ, 5
- codage, 43
 - de longueur fixe, 43
 - préfixe, 44
- complexité, 3, 4
 - d'un algorithme de tri, 19
 - dans le meilleur des cas, 4
 - dans le pire des cas, 4
 - en moyenne, 4
- descendant, 11
- enregistrement, 5
- feuille, 11
- FIFO, 9
- file, 9
- hachage, 36
 - avec chaînage externe, 41
 - avec chaînage interne, 39
 - linéaire, 38
- hauteur, 11
- instance, 3
- LIFO, 8
- liste, 6
 - chaînée, 7
- niveau, 11
- nœud interne, 11
- opération élémentaire, 3
- parcours, 12
 - en ordre infixe, 14
 - en ordre milieu, 14
 - en ordre postfixe, 12
 - en ordre préfixe, 12
 - en ordre suffixe, 12
 - en ordre symétrique, 14
 - en postordre, 12, 14
 - en préordre, 12, 14
- partition, 23
- pile, 8
- pointeur, 5
- profondeur, 11
- queue, 6
- racine, 10
- recherche, 18
 - dichotomique, 18
 - séquentielle, 1, 18
- référence, 5
- sous-arbre, 10
- structure, 5
 - de données, 5
 - linéaire, 6
- tableau, 5
- tas, 28
- tête, 6
- tri, 18
 - comparatif, 20
 - insertion, 22
 - rapide, 23
 - sélection, 22
 - tas, 32

Bibliographie

- A. Aho, J. Hopcroft, J. Ullman, *Structures de données et algorithmes*, InterEditions, Paris, 1987.
- A. Aho, J. Ullman, *Concepts fondamentaux de l'informatique*, Dunod, Paris, 1993.
- D. Beauquier, J. Berstel, P. Chrétienne, *Éléments d'algorithmique*, Masson, Paris, 1992.
- T. Cormen, C. Leiserson, R. Rivest, *Introduction à l'algorithmique*, Dunod, Paris, 1994.
- M.-C. Gaudel, M. Soria, C. Froidevaux, *Types de données et algorithmes*, McGraw Hill, Paris, 1990.
- D.E. Knuth, *The art of computer programming*, Reading, Massachusetts, Addison-Wesley, 1968.
- N. Wirth, *Algorithmes et structures de données*, Eyrolles, Paris, 1987.